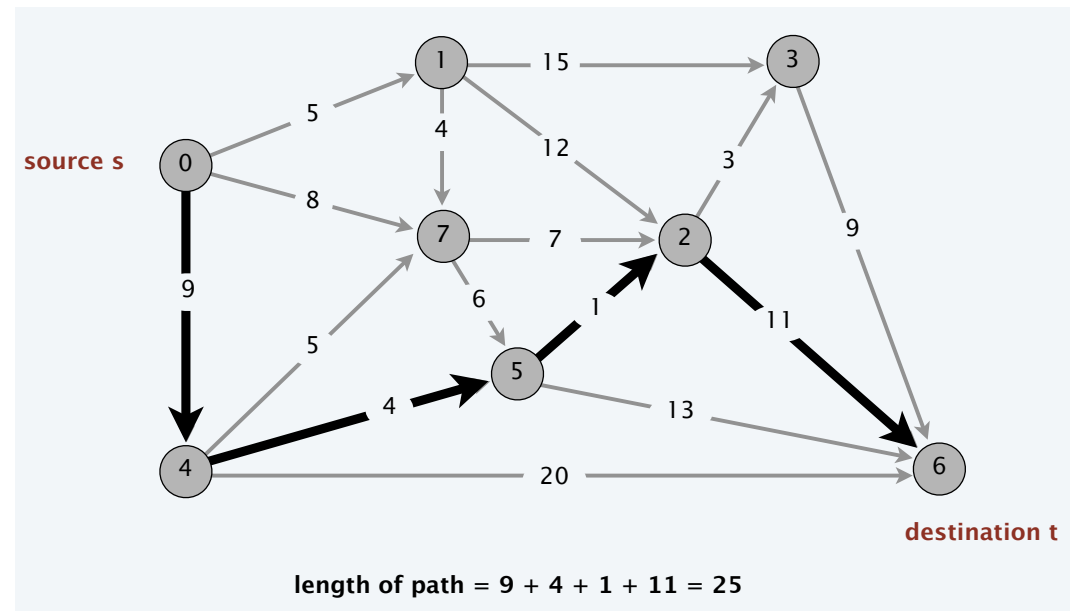


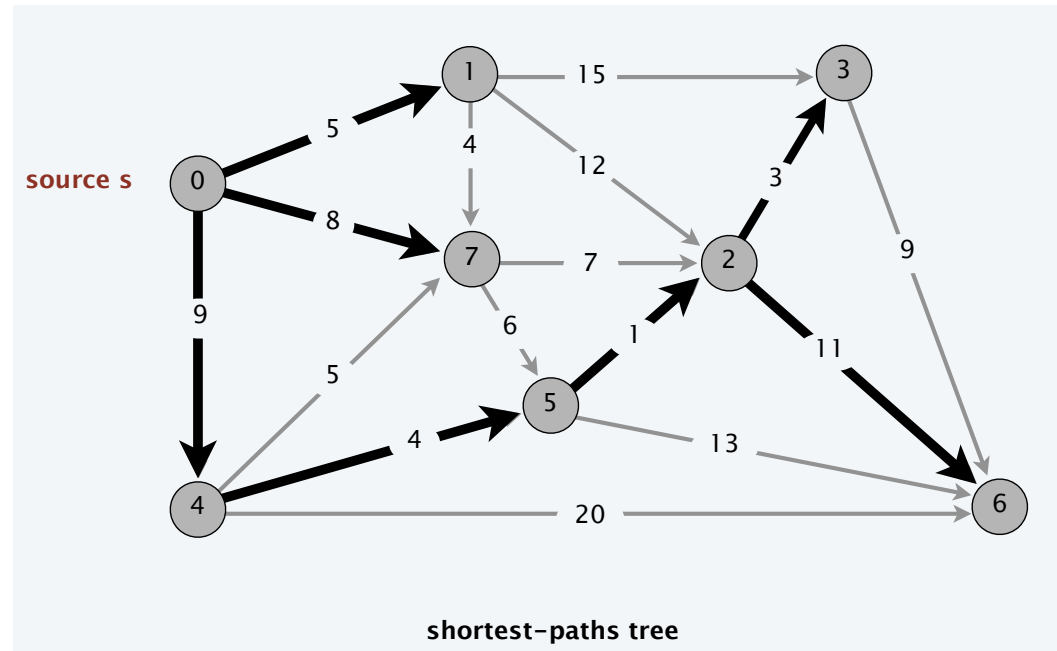
What is a Shortest Path?

- Given weighted, directed graph $G(V,E)$ (weights on edges)...
 - ...what is shortest path from vertex s to t ?
 - Distance is additive
- Applications
 - Google maps
 - Routing packets on the Internet
 - Social networks



Single Source Shortest Paths (SSSP)

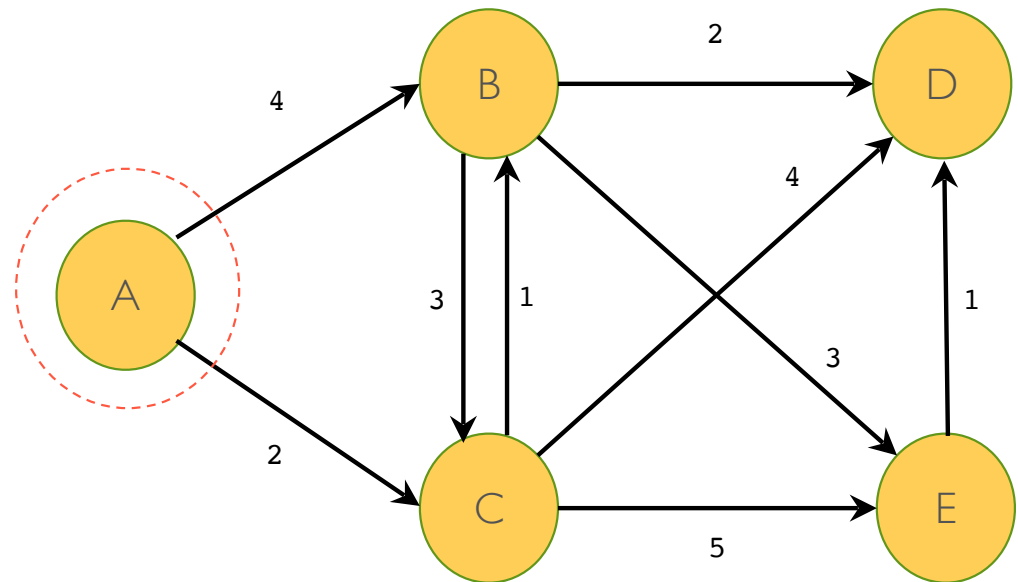
- Given a graph and a source vertex
 - find the shortest paths to all other vertices
 - results in a shortest-path tree
 - Single directed path to every other vertex



Quick Insight Quiz

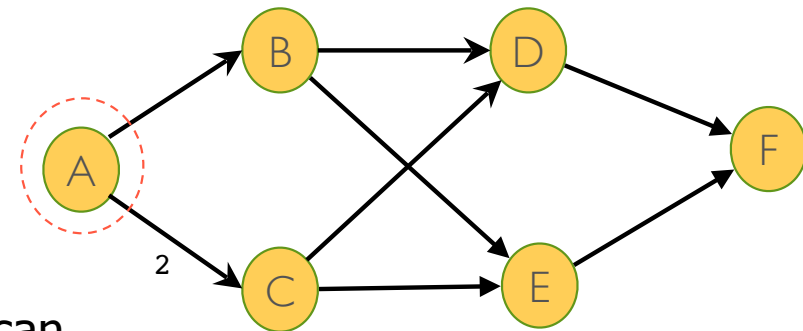
- Suppose that you change the length of every edge of G as follows.
For which is every shortest path in G a shortest path in G' ?

- Add 17
- Multiply by 17
- Either A or B
- Neither A nor B



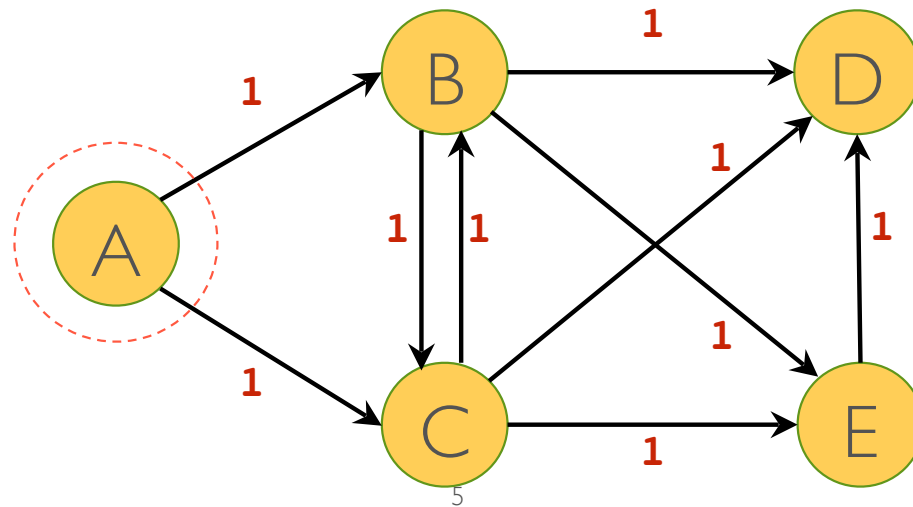
Important Property of Shortest Paths

- **Lemma:** The shortest path from vertex s to a vertex t is composed of shortest paths to and from any intermediate vertices
 - Proof by contradiction on path from s to t
 - Consider intermediate vertex v
 - If better path from s to v was possible, then can replace original path to v and get shorter
 - If better path from v to t was possible, then can original path from v to t and get shorter
 - Hence, original path was not shortest path
- **Bellman's Principle of Optimality**



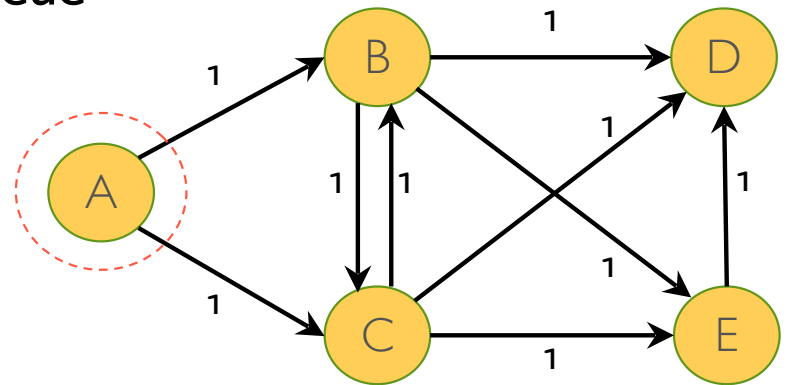
Simpler Problem: Unit Edges

- Let's start with simpler problem
 - On graph where every edge has unit cost
 - Shortest path = minimum number of hops



Solution: Breadth-First Search

- Use BFS to find shortest path from A to E.
 - Consider all steps of adding/removing nodes from queue ...
 - ... updating each vertex's predecessor
 - predecessor: id of vertex that adds current vertex to queue
 - ... updating distances when added to queue
 - Distance to vertex is 1 more than distance to vertex that added current vertex to queue



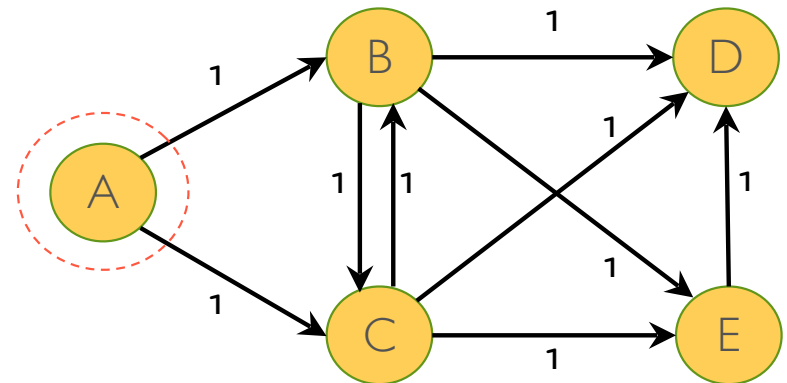
Example

Queue
A

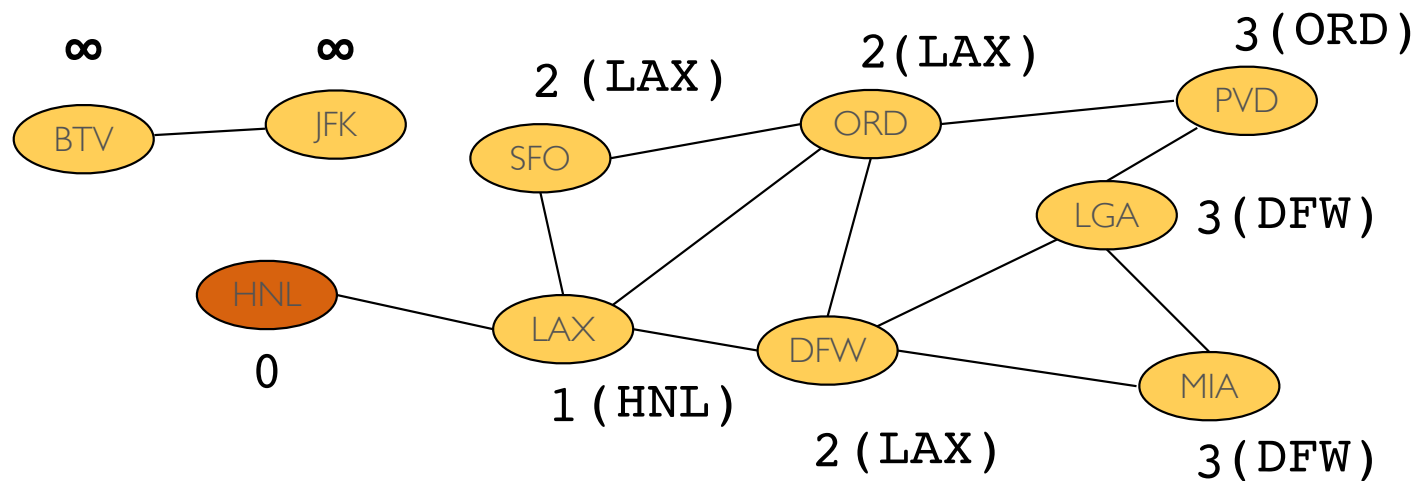
Queue
B
C

Queue
C
D
E

Queue
E



numStops (with BFS)

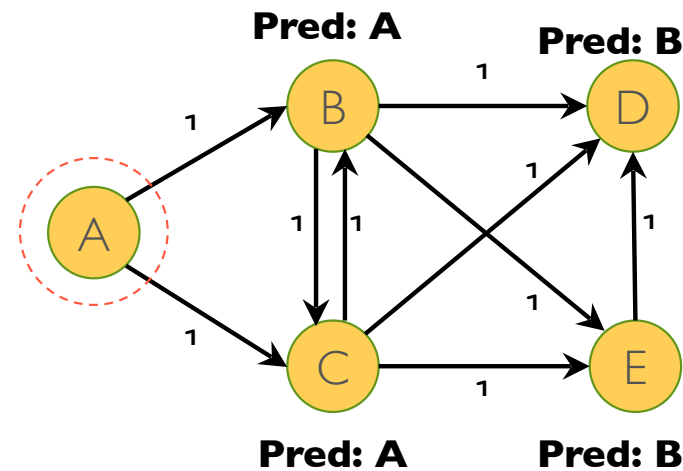
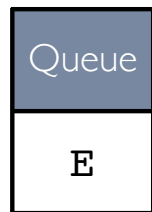


Have *distance*
Want a *path*

Follow 'predecessor' pointers
to get a path

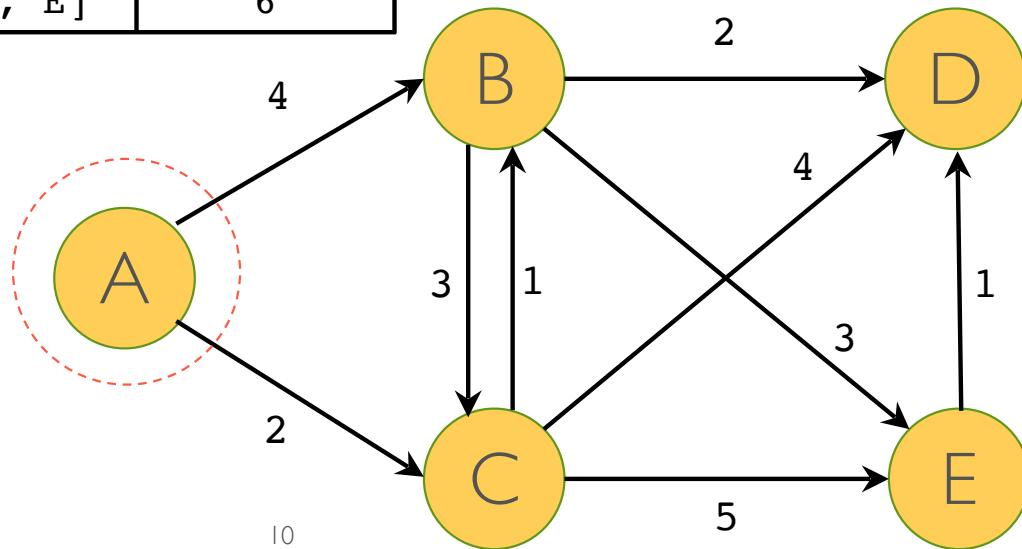
Breadth First Search

- When we dequeue E...
- ...we traverse the pred to return paths
 - shortest path to E: [A,B,E]

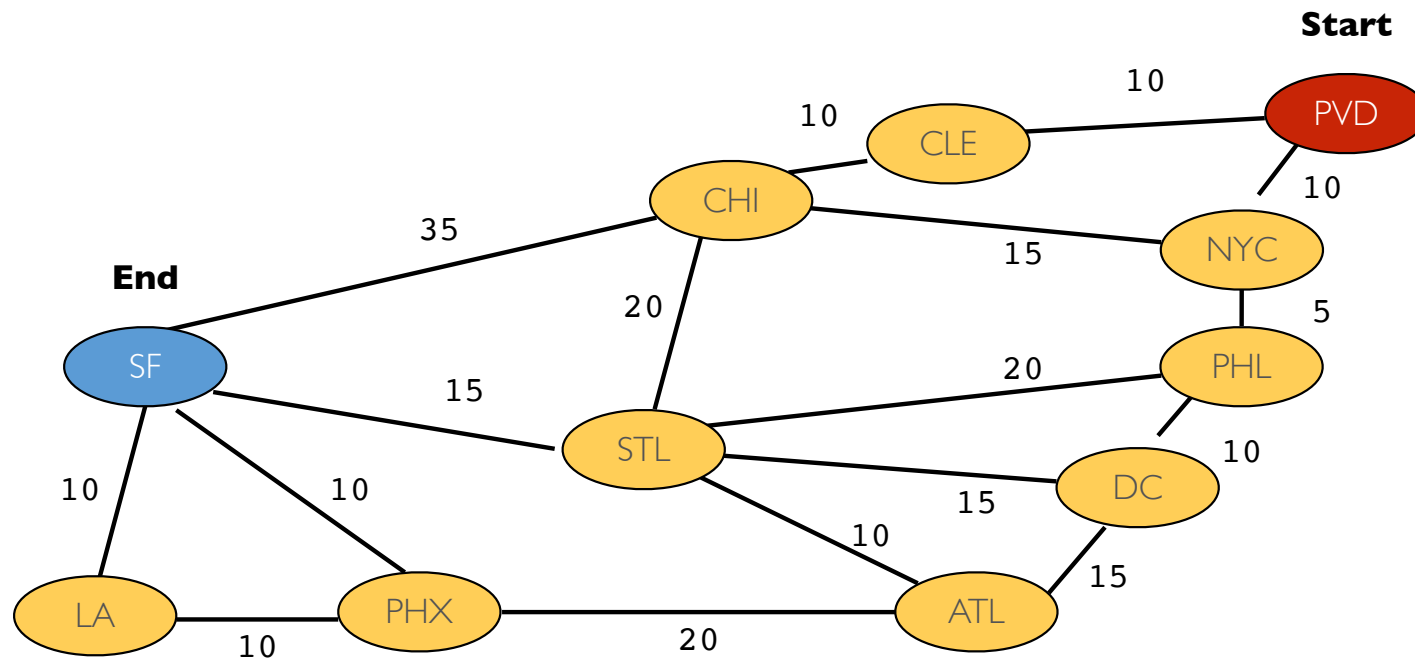


Non-unit Edge Weights

Goal Node	Shortest Path	Shortest Distance
B	[A, C, B]	3
C	[A, C]	2
D	[A, C, B, D]	5
E	[A, C, B, E]	6



Our Graph

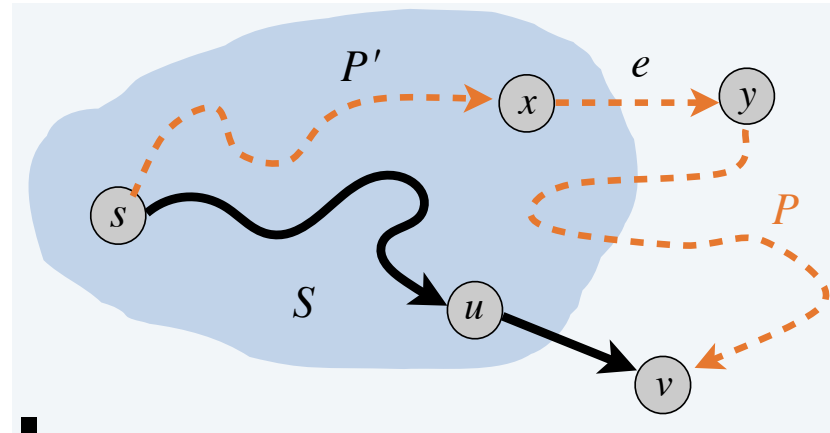


Dijkstra's Algorithm

- Greedy Algorithm: Assumes all edges have **nonnegative** weights
 - Maintain a set of **explored** nodes S for which algorithm has determined $D[u]$ = length of a shortest s to u path
 - Initialize $S = \{s\}$, $D[s] = 0$; $D[v] = \text{infinity}$
 - Choose unexplored node $v \notin S$ which minimizes $D[u] + w(u, v)$, $u \in S$
Set $D[v] = \min\{\min_{\{(u,v) \in E: u \in S\}} D[u] + w(u, v), D(v)\}$, and add v to S
Set $\text{pred}[v] = \text{vertex } u \text{ in } S \text{ that achieves } d[v]$
 - Repeat until all vertices are explored, so $S = V$
 - Path to any vertex can be found by using $\text{pred}[]$ labels

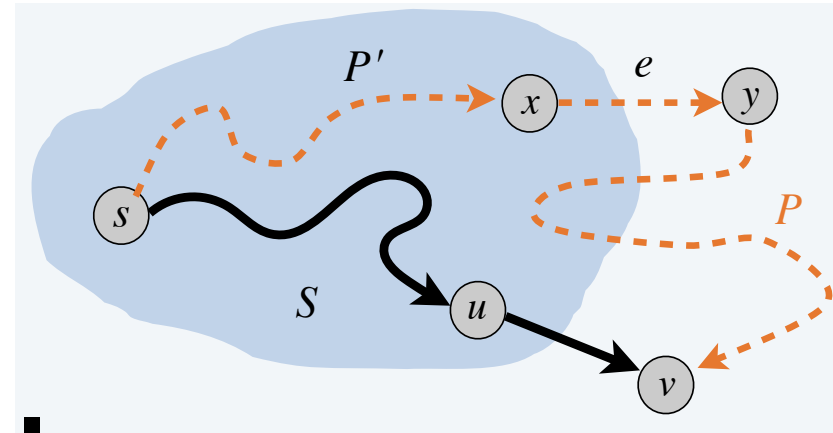
Dijkstra's Algorithm: Correctness

- Claim: For every vertex v in S , $D[v]$ is shortest path distance from s to v
 - Prove by induction: clearly true when $S = \{s\}$, $D[s] = 0$
 - Assume true for S of size k . Let v be the next vertex added to S , and let (u,v) be the final edge, so $D[v] = D[u] + w(u,v)$, where u in S
 - Assume there is another path P' from s to v that is shorter than $D[v]$
 - P' starts in S , and must leave S to go to some vertex $x \in S$, then use an edge e that leaves S for a vertex y on the way to v
 - Note: y can be v , x can be u , but not both!



Dijkstra's Algorithm: Correctness - 2

- Claim: For every vertex v in S , $D[v]$ is shortest path distance from s to v
 - Let $L(P')$ be the length of P' , and let P be the path in P' from y to v
Then, $L(P') = D[x] + w(e) + L(P) \geq D[x] + w(e)$ because all weights are nonnegative, so $L(P) \geq 0$
 - Note: $D[x] + w(e) \geq D[u] + w(u, v)$ because we chose (u, v) such that
$$D[v] = \min_{\{(u,v) \in E: u \in S\}} D[u] + w(u, v)$$
 - Therefore, $D[v]$ is no longer than $L(P')$, which was shortest path distance, so $D[v]$ is the shortest path distance



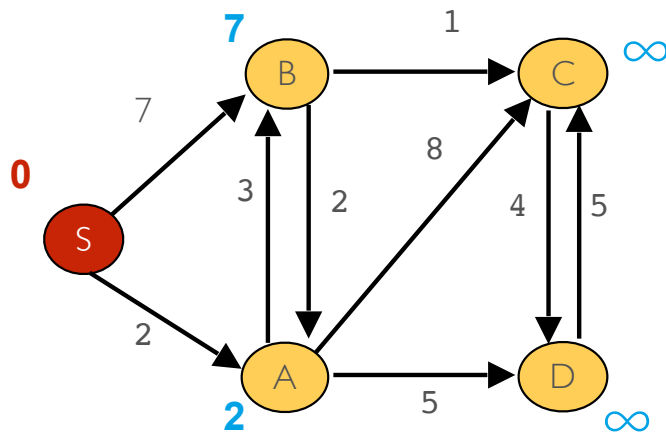
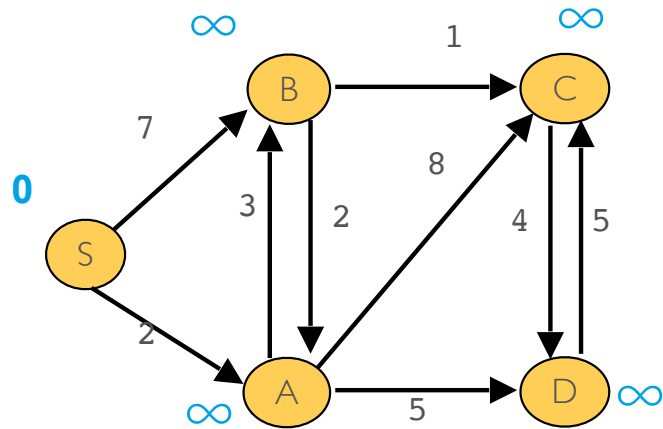
Implementing Dijkstra's Algorithm

- Previous description is hard to implement
- Better approach: Keep a priority heap of vertices v not in S , with keys $D[v]$ that are shortest distance to v of all paths through S with one edge leaving S for v
 - $D[v]$ is an upper bound on shortest distance to v if v not in S
 - Best node v not in S is at top of priority queue
- Can maintain $D[v]$ to all v not explored recursively as we increase S

Dijkstra: Pseudo code

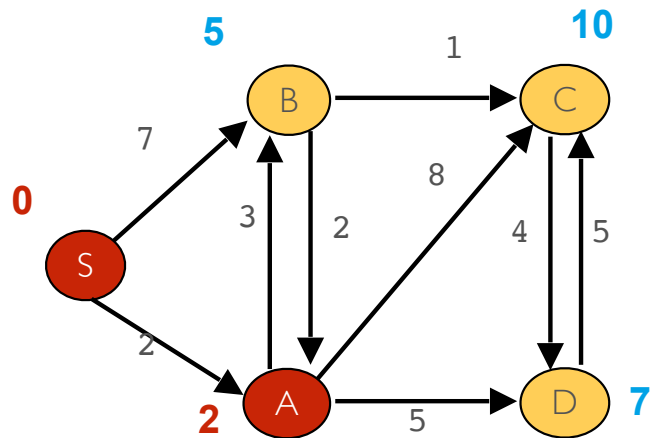
- Initially, $S = \emptyset$, $D(s) = 0$, $D[v] = \infty$, $v \neq s$, and $\text{pred}[v] = \text{null}$ for all $v \in V$
- Insert all $v \in V$ into priority queue PQ indexed by $D[v]$
- While PQ is not empty,
 - Delete_min of PQ, denoted by u , and add u to S
 - For each edge (u,v) leaving S :
 - If $D[v] > D[u] + w(u, v)$:
 - Decrease_key of v in PQ to $D[u] + w(u, v)$
 - Set $\text{pred}[v] = u$, $D[v] = D[u] + w(u, v)$

Dijkstra's Algorithm Example

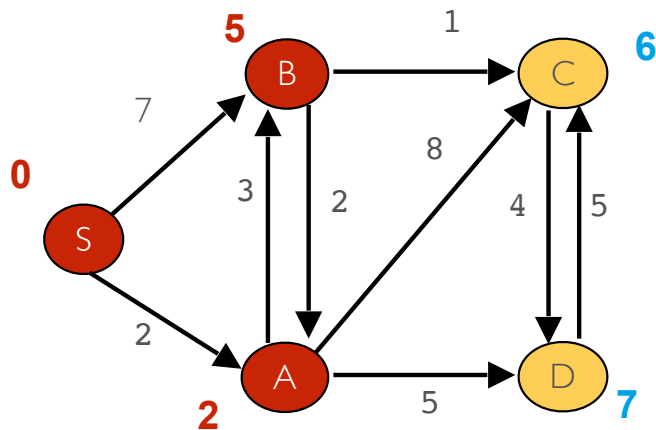


- Step 1
 - Label source s w/ distance 0
 - Label other vertices w/ distance ∞
 - Add all nodes to Q
- Step 2
 - Remove vertex n with min. priority from Q (n=s first)
 - Calculate distance from source to removed node's neighbors...
 - ...by adding adjacent edge weights to distance $D[n]$
 - Reduce keys if $D[v] > D[n] + w(n,v)$, set new $D[v] = D[n] + w(n,v)$, update $\text{pred}(v) = n$

Example - 2

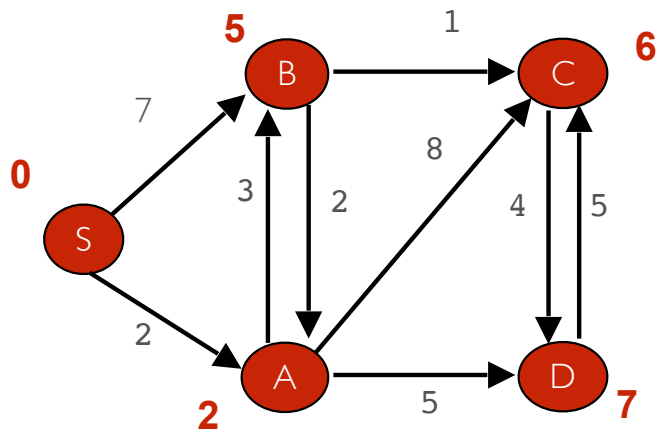
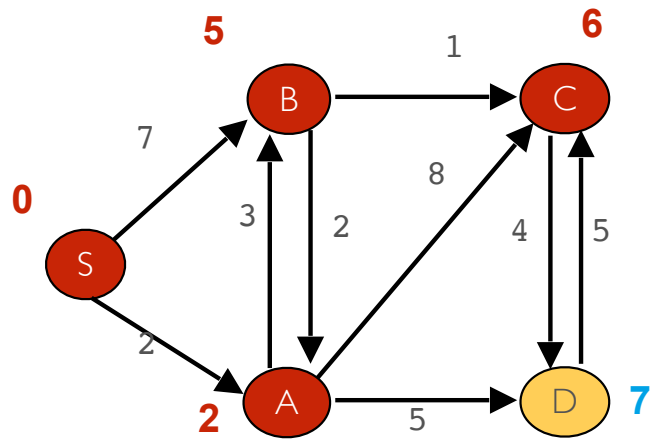


- Step 3
 - While Q isn't empty,
 - repeat previous step
 - removing A this time
 - Priorities of nodes in Q may have to be updated
 - ex: B's priority



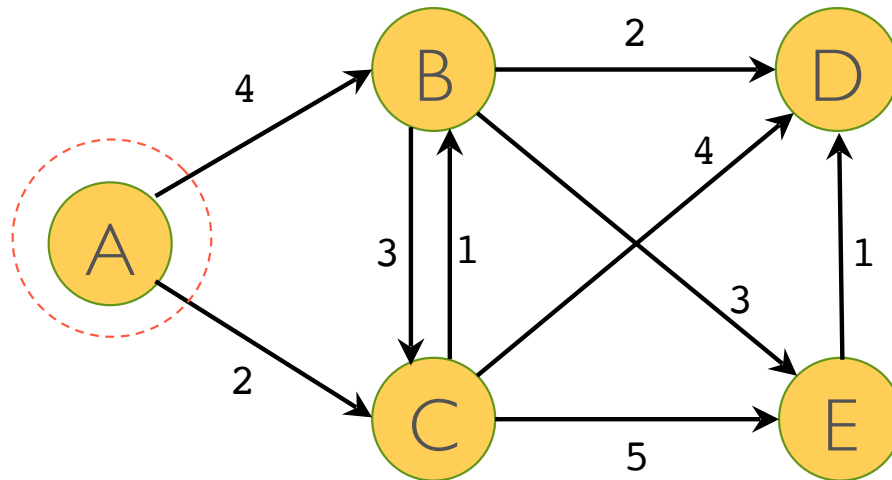
- Step 4
 - Repeat again by removing vertex B
 - Update distances that are shorter using this path than before
 - ex: C now has a distance 6 not 10

Example - 3



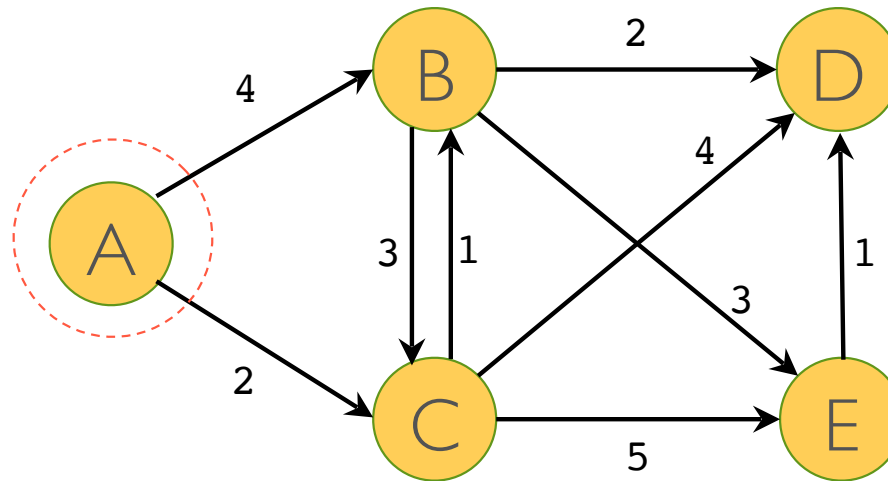
- Step 5
 - Repeat
 - this time removing C
- Step 6
 - After removing D...
 - ...every node has been visited...
 - ...and marked with shortest distance to source

Another Example



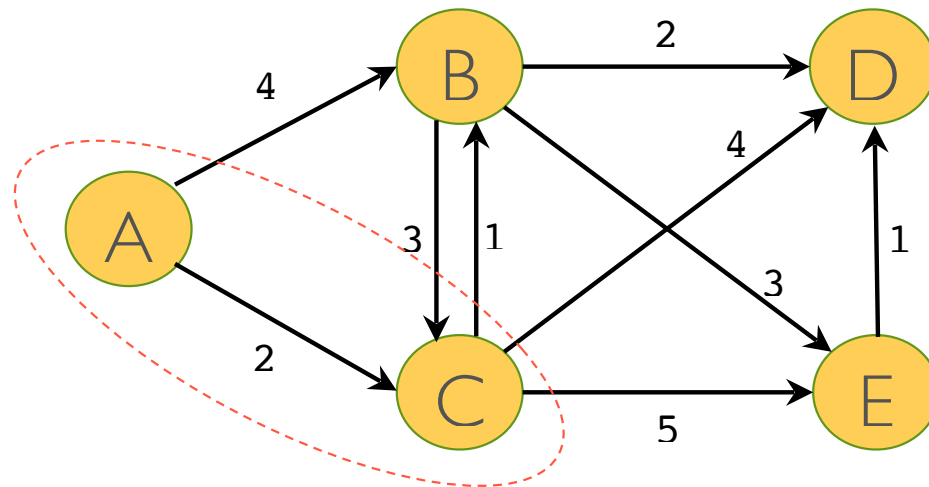
	A	B	C	D	E
D[]	0	∞	∞	∞	∞
pred[]					

Another Example



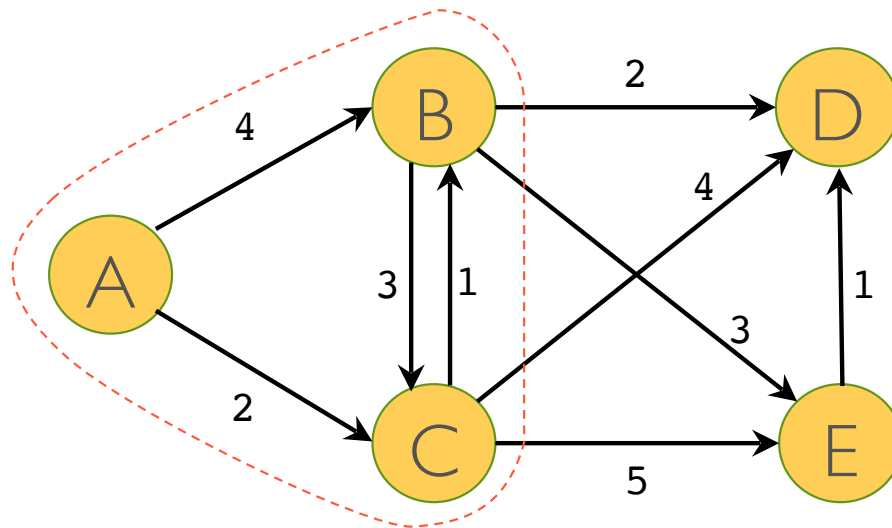
	A	B	C	D	E
D[]	0	4	2	∞	∞
pred[]	Null	A	A	Null	Null

Another Example



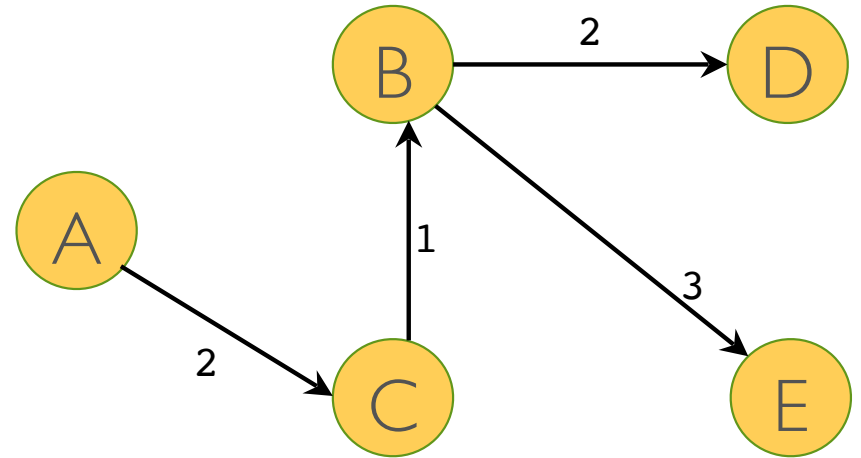
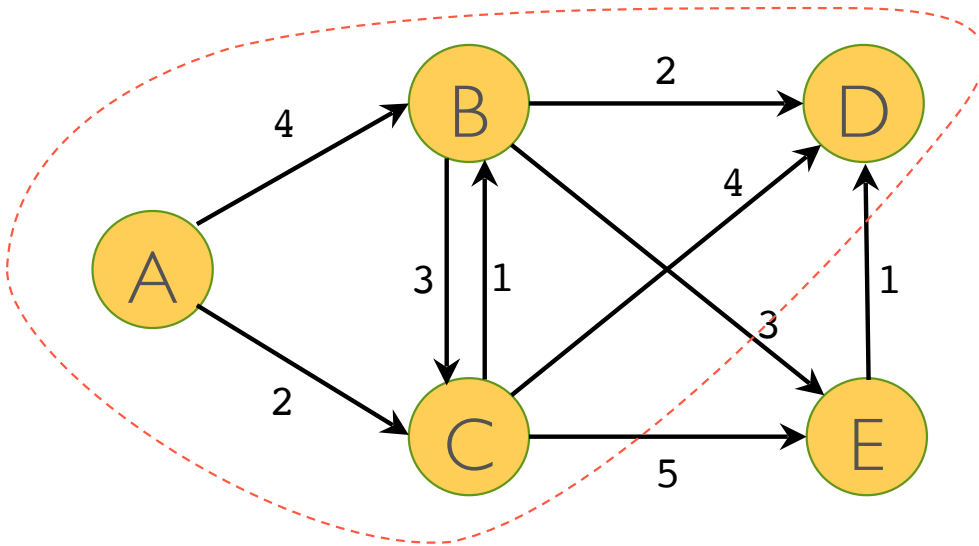
	A	B	C	D	E
D[]	0	3	2	6	7
pred[]	Null	C	A	C	C

Another Example



	A	B	C	D	E
D[]	0	3	2	5	6
pred[]	Null	C	A	B	B

Another Example



	A	B	C	D	E
D[]	0	3	2	5	6
pred[]	Null	C	A	B	B

Dijkstra vs Prim

```
function dijkstra(G, s):
    // Input: weighted, dir. graph G, vertices V, source s
    // Output: Nothing
    // Purpose: compute for all V shortest distance from s
    for v in V:
        v.dist = infinity // Initialize distance
        v.prev = null     // Initialize pred pointers to
null
    s.dist = 0           // Set distance to start to 0

    PQ = PriorityQueue(V) // Use v.dist as priorities
    while PQ not empty:
        u = PQ.removeMin()
        for all edges (u, v): //each edge coming out of u
            if v.dist > u.dist + weight(u, v): //
                v.dist = u.dist + weight(u,v) // Replace
                v.prev = u // Maintain pointers for path
                PQ.decreaseKey(v, v.dist)
```

```
function prim(G):
    // Input: weighted undir. graph G with vertices V
    // Output: list of edges in MST
    for all v in V:
        v.dist = infinity
        v.prev = null
    s = any v in V // pick a random source s
    s.dist = 0
    MST = []
    PQ = PriorityQueue(V) // priorities will be v.cost
    while PQ is not empty:
        U = PQ.removeMin()
        if u.prev != null:
            MST.append((u, u.prev))
        for all edges (u,v) such that v is in PQ:
            if v.dist > weight(u,v):
                v.dist = weight(u,v)
                u.prev = v
                PQ.decreaseKey(v, v.dist)
    return MST
```

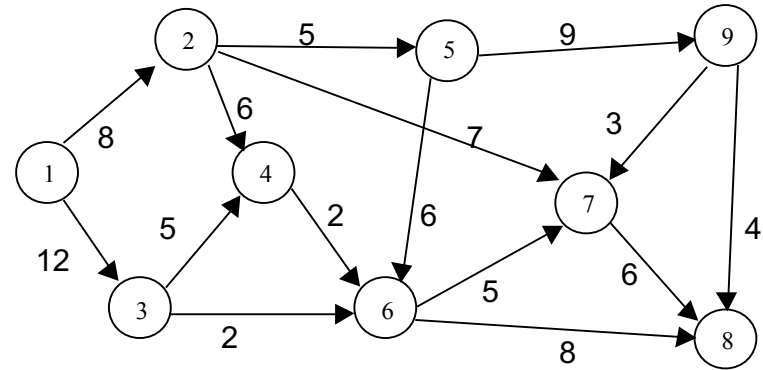
Dijkstra: Which Priority Queue?

- Algorithm complexity
 - Priority queue with $|V|$ elements
 - Examine every edge once
 - May Decrease_key for every edge
 - $|V|$ Delete_min
- Total run time with array as priority queue: $O(|E| + |V|^2)$
- Total run time with binary heap as priority queue: $O((|E| + |V|)\log(|V|))$
- Total run time with Fibonacci heap or rank-pairing heap as priority queue: $O(|E| + |V|\log(|V|))$

Example

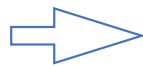
- Source node 1, use binary heap
 - $Q[]$ is position in heap

If $D[v] > D[u] + w(u, v)$, set $D[v] = D[u] + w(u, v)$



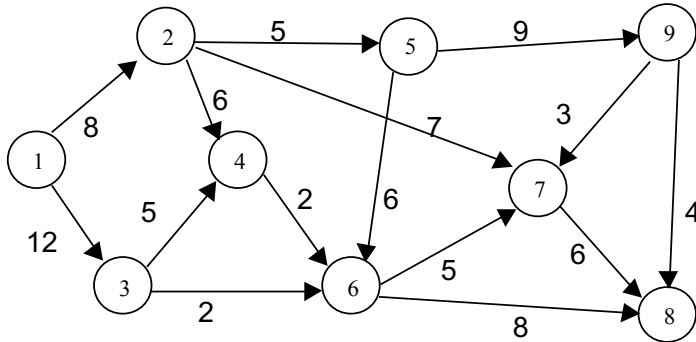
	D[]	pred[]	Q[]
1	0	Null	0
2	Inf	Null	
3	Inf	Null	
4	Inf	Null	
5	Inf	Null	
6	Inf	Null	
7	Inf	Null	
8	Inf	Null	
9	Inf	Null	

Pop 1



	D[]	pred[]	Q[]
1	0	Null	
2	8	1	0
3	12	1	1
4	Inf	Null	
5	Inf	Null	
6	Inf	Null	
7	Inf	Null	
8	Inf	Null	
9	Inf	Null	

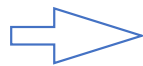
Example - 2



If $D[v] > D[u] + w(u, v)$, set $D[v] = D[u] + w(u, v)$

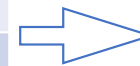
	D[]	pred[]	Q[]
1	0	Null	
2	8	1	0
3	12	1	1
4	Inf	Null	
5	Inf	Null	
6	Inf	Null	
7	Inf	Null	
8	Inf	Null	
9	Inf	Null	

Pop 2



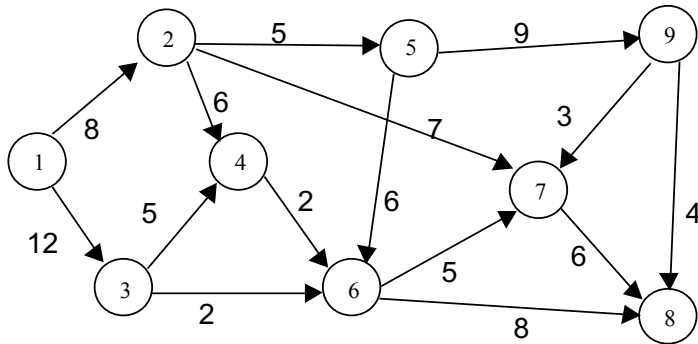
	D[]	pred[]	Q[]
1	0	Null	
2	8	1	
3	12	1	0
4	14	2	1
5	13	2	2
6	Inf	Null	
7	15	2	3
8	Inf	Null	
9	Inf	Null	

Pop 3



	D[]	pred[]	Q[]
1	0	Null	
2	8	1	
3	12	1	
4	14	2	1
5	13	2	0
6	14	3	3
7	15	2	2
8	Inf	Null	
9	Inf	Null	

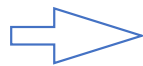
Example - 3



If $D[v] > D[u] + w(u, v)$, set $D[v] = D[u] + w(u, v)$

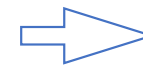
	D[]	pred[]	Q[]
1	0	Null	
2	8	1	
3	12	1	
4	14	2	1
5	13	2	0
6	14	3	3
7	15	2	2
8	Inf	Null	
9	Inf	Null	

Pop 5



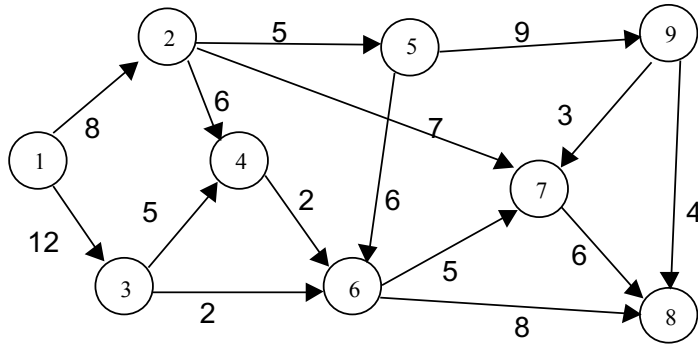
	D[]	pred[]	Q[]
1	0	Null	
2	8	1	
3	12	1	
4	14	2	1
5	13	2	
6	14	3	0
7	15	2	2
8	Inf	Null	
9	22	5	3

Pop 6



	D[]	pred[]	Q[]
1	0	Null	
2	8	1	
3	12	1	
4	14	2	0
5	13	2	
6	14	3	
7	15	2	2
8	22	6	3
9	22	5	1

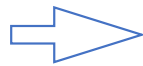
Example - 4



If $D[v] > D[u] + w(u, v)$, set $D[v] = D[u] + w(u, v)$

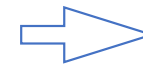
	D[]	pred[]	Q[]
1	0	Null	
2	8	1	
3	12	1	
4	14	2	0
5	13	2	
6	14	3	
7	15	2	2
8	22	6	3
9	22	5	1

Pop 4



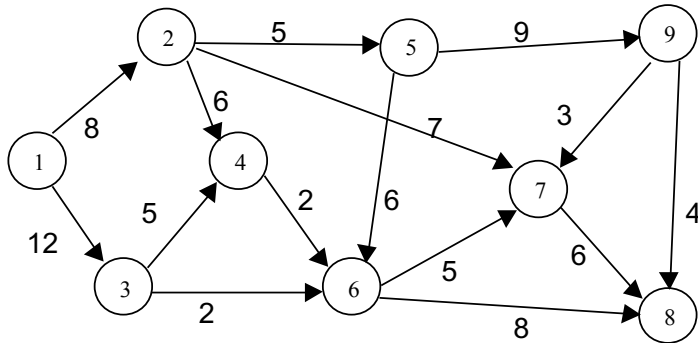
	D[]	pred[]	Q[]
1	0	Null	
2	8	1	
3	12	1	
4	14	2	
5	13	2	
6	14	3	
7	15	2	0
8	22	6	2
9	22	5	1

Pop 7



	D[]	pred[]	Q[]
1	0	Null	
2	8	1	
3	12	1	
4	14	2	
5	13	2	
6	14	3	
7	15	2	
8	21	7	0
9	22	5	1

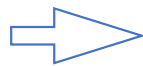
Example - 5



If $D[v] > D[u] + w(u, v)$, set $D[v] = D[u] + w(u, v)$

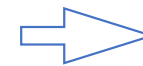
	D[]	pred[]	Q[]
1	0	Null	
2	8	1	
3	12	1	
4	14	2	
5	13	2	
6	14	3	
7	15	2	
8	21	7	0
9	22	5	1

Pop 8



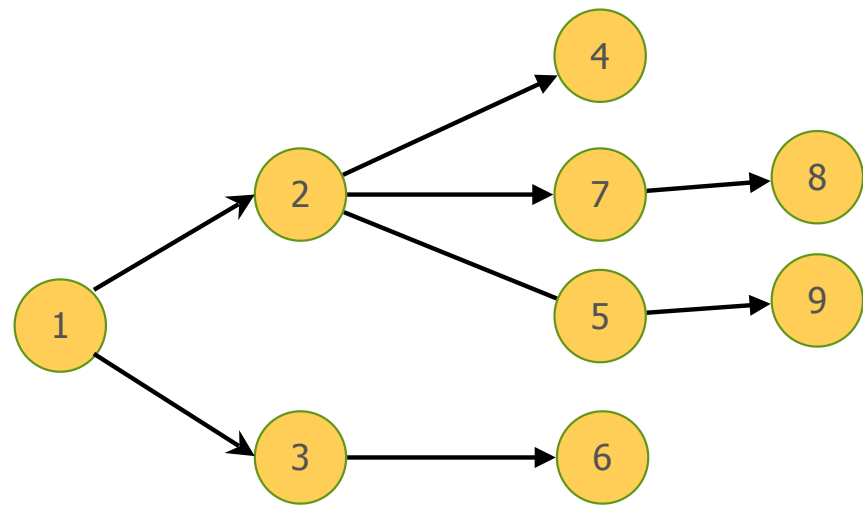
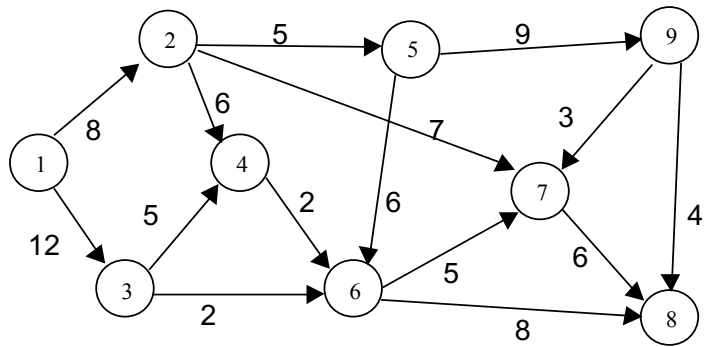
	D[]	pred[]	Q[]
1	0	Null	
2	8	1	
3	12	1	
4	14	2	
5	13	2	
6	14	3	
7	15	2	
8	21	7	
9	22	5	0

Pop 9



	D[]	pred[]	Q[]
1	0	Null	
2	8	1	
3	12	1	
4	14	2	
5	13	2	
6	14	3	
7	15	2	
8	21	7	
9	22	5	

Example - 6



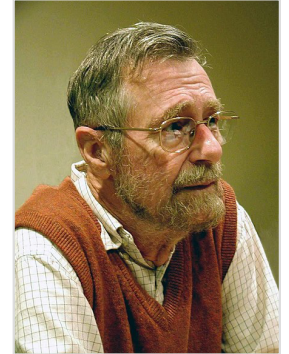
Historical Aside

- Edsger Dijkstra (1930-2002): Dutch computer scientist

- “What’s the shortest way to travel from Rotterdam to Groningen? It is the algorithm for the shortest path, which I designed in about 20 minutes. One morning (1956) I was shopping in Amsterdam with my young fiancée, and tired, we sat down on the café terrace to drink a cup of coffee and I was just thinking about whether I could do this, and I then designed the algorithm for the shortest path...

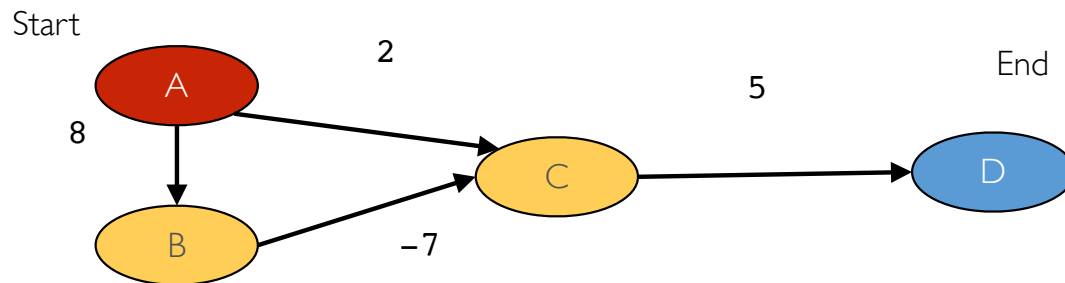
One of the reasons that it is so nice was that I designed it without pencil and paper. I learned later that one of the advantages of designing without pencil and paper is that you are almost forced to avoid all avoidable complexities.” (Communications of the ACM 53 (8), 2001)

- Won the Turing award in 1972
- Dijkstra’s algorithm used in internet protocols: OSPF (Open Shortest Path First) and IS-IS (Intermediate System to Intermediate System) protocols
- Many other contributions...e.g. concurrent programming and smoothsort



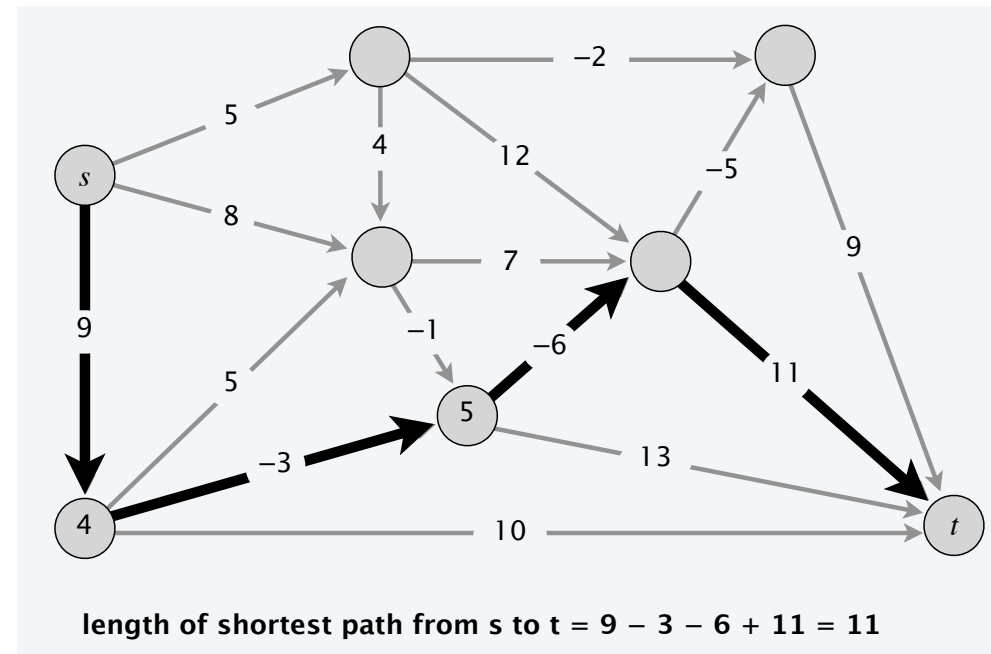
Dijkstra Limitation

- Edges must have non-negative weight
 - Otherwise, proof of correctness fails (continuation of path can have negative length)
- For graph below, Dijkstra computes shortest A-D of 7



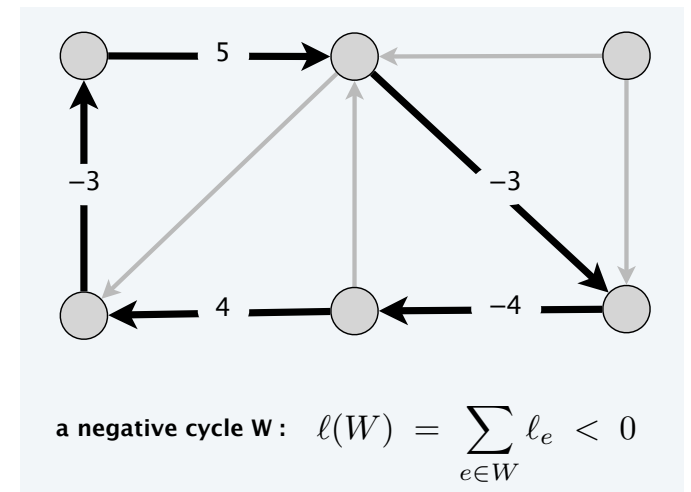
Shortest paths: Graphs with Negative Weights

- Need different algorithm
- Dijkstra's algorithm: "label-setting"
 - Once vertex exits priority queue, its distance label never changes
- Different class of algorithms: "label-correcting"
 - Exploit Bellman's principle of optimality
 - Will handle some cases with negative weights



Limitations: Negative Cycles

- A negative cycle is a directed cycle where the sum of the weights on the edges is negative
- **Lemma:** if some path from s to t contains a negative cycle, there does not exist a shortest path from s to t
 - Can go around the negative cycle an infinite amount of time...
- **Lemma:** if there exist no negative cycles in directed graph, then there exists a shortest path from s to t that is simple (no cycles) and has $< |V|$ edges
 - Consider minimum number of edges among all shortest paths from s to t
 - If number of edges $\geq |V|$, then it must have a cycle
 - If this path has a cycle, can remove cycle and get one with less edges and same total weight



Dynamic Programming

- **Dynamic programming**: an algorithmic technique for solving an optimization problem
 - Breaking it down into simpler subproblems
 - Exploit that the optimal solution to the overall problem depends upon the optimal solution to its subproblems
 - Bellman's principle of optimality: shortest path from s to t is composed of shortest paths to and from any intermediate vertices
- Idea: let's solve subproblem of finding shortest path with a maximum number of edges

Dynamic Programming

- **Dynamic programming**: an algorithmic technique for solving an optimization problem involving sequences of decisions by
 - Breaking it down into simpler subproblems
 - Exploiting that the optimal solution to the overall problem depends upon the optimal solution to its subproblems
 - Bellman's principle of optimality: shortest path from s to t is composed of shortest paths to and from any intermediate vertices
- Idea: let's solve subproblem of finding shortest path with a maximum number of edges
 - Easy if we limit it to 1; if we extend it to $|V|-1$, we solved the full problem

EC504 ALGORITHMS AND DATA STRUCTURES
SPRING 2022 MONDAY & WEDNESDAY
2:30 PM - 4:15 PM

Prof: David Castañón, dac@bu.edu

Teaching Assts:

Yangruirui Zhou: yrrzhou@bu.edu

Junyi Tang: jytang2@bu.edu

Yiqin Zhang: evezhang@bu.edu

Bellman-Ford Algorithm

- **Notation:** $D(k,v)$ is shortest distance from vertex s to vertex v on paths with less than or equal to k edges

Initial value: $D(0,v) = \infty, v \neq s; 0, v = s$

Bellman equation $D(k, v) = \min \left\{ D(k-1, v), \min_{(u,v) \in E} [D(k-1, u) + w(u, v)] \right\}$ (DP)

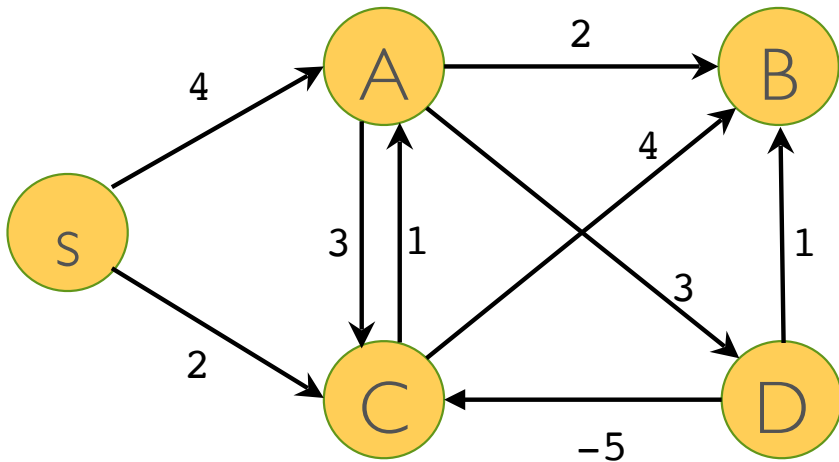
- Bellman-Ford Algorithm
 - For each $v \in V$, set $D(0,v) = \infty, v \neq s; 0, v = s$; set $\text{pred}[v] = \text{null}$
 - For $k = 1$ to $|V| - 1$:
 - For each $v \in V$, set $D(k, v) = D(k-1, v)$
 - For each $(u, v) \in E$:
 - if $D(k, v) > D(k-1, u) + w(u, v)$:
 - Set $D(k, v) = D(k-1, u) + w(u, v)$, set $\text{pred}[v] = u$

Bellman-Ford Algorithm Properties

- $D(k,v)$ is the shortest distance for a path of k or less edges from s to v
 - Prove by induction: obviously true at $k = 0$. Assume true for $k-1$. Assume $D(k, v) = D(k-1, v)$. Then, from the iteration, no path with k vertices is shorter than $D(k,v)$. Note that $D(k-1, u) + w(u, v)$ is the shortest distance of all paths of length k or less that end in v and have next to last vertex u . This proves the result, because $D(k, v) = \min_{(u,v) \in E} D(k-1, u) + w(u, v)$
- At each iteration, $D(k,v)$ is upper bound on the shortest path distance from s to v
- $D(k,v)$ is non-increasing in k
- If $D(k-2, v) = D(k-1, v)$, one does not have to scan edges from node v for $D(k,v)$
 - Best path to v is less than $k-1$ edges
- Unlike Dijkstra's algorithm, each edge is examined more than once; once per k
- Brute force complexity: $|V|$ iterations, each of which is $O(|E|)$ $\rightarrow$ complexity $O(|V||E|)$

Bellman-Ford Algorithm Properties

- Algorithm converges to shortest paths provided no negative cycles exist in graph
- If negative cycles exist, predecessor graph will have cycles!
- Distances continue decreasing after $|V|-1$ iterations (neg. cycle detection)



	D(s)	D(A)	D(B)	D(C)	D(D)
k=1	0	4	Inf	2	Inf
k=2	0	3	6	2	7
k=3	0	3	5	2	6
k=4	0	3	5	1	6

Final pred array: pred[s] = null, pred[A] = C, pred[B] = A, pred[C] = D, pred[D] = A

Better Bellman-Ford Implementation

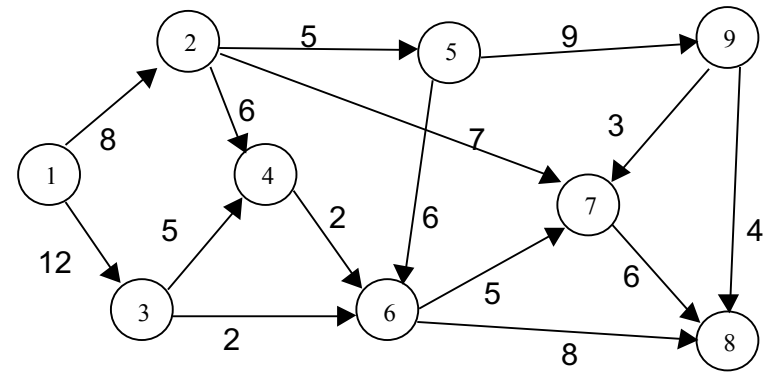
- Use a queue of vertices where distances have changed
 - Avoid looking at edges out of vertices where distances have not changed
- Algorithm
 - Initialize $D[s] = 0$; $D[v] = \infty, v \neq s$; $\text{pred}[v] = \text{null}$
 - Insert s into queue Q ; mark $\text{inqueue}[s] = \text{true}$, mark $\text{inqueue}[v] = \text{false}, v \neq s$
 - While Q is not empty:
 - Select u out of queue, mark $\text{inqueue}[u] = \text{false}$
 - For each edge (u,v) in E :
 - If $D[v] > D[u] + w(u, v)$:
 - Set $D[v] = D[u] + w(u, v)$, set $\text{pred}[v] = u$
 - if $\text{inqueue}[v] = \text{false}$, add v to Q , mark $\text{inqueue}[v] = \text{true}$

Better Bellman-Ford Implementation

- Many variations possible
 - Have not specified in what order to insert vertices into queue Q
 - Have not specified in what order to extract vertices from Q
 - Algorithm works anyway, as long as there are no negative weight cycles
- Detecting negative cycles
 - A negative cycle exists if and only if a vertex enters the Q $|V|$ times
- Simplest implementation: use queue in FIFO order
- More interesting implementations: insert vertices with shorter distances $D[v]$ in the front of the queue
- Convergence happens because distance labels $D[v]$ decrease monotonically, and are bounded below
- Bellman-Ford and variations are known as label-correcting algorithms: distance labels $D[v]$ are not set when v leaves the Queue
 - Less overhead than Dijkstra; can be empirically faster

Example

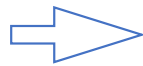
- Source node 1



If $D[v] > D[u] + w(u, v)$, set $D[v] = D[u] + w(u, v)$

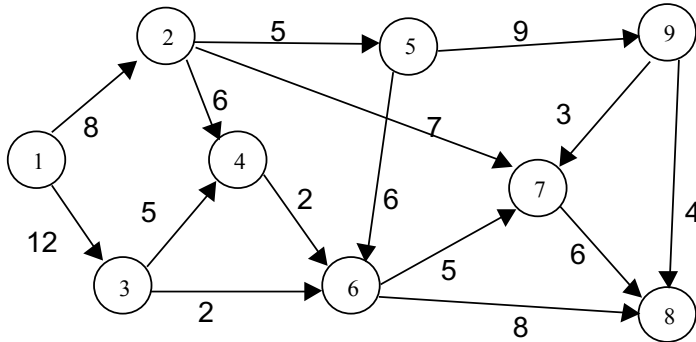
	D[]	pred[]	Q[]
1	0	Null	1
2	Inf	Null	
3	Inf	Null	
4	Inf	Null	
5	Inf	Null	
6	Inf	Null	
7	Inf	Null	
8	Inf	Null	
9	Inf	Null	

Pop 1



	D[]	pred[]	Q[]
1	0	Null	
2	8	1	1
3	12	1	2
4	Inf	Null	
5	Inf	Null	
6	Inf	Null	
7	Inf	Null	
8	Inf	Null	
9	Inf	Null	

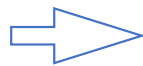
Example - 2



If $D[v] > D[u] + w(u, v)$, set $D[v] = D[u] + w(u, v)$

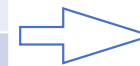
	D[]	pred[]	Q[]
1	0	Null	
2	8	1	1
3	12	1	2
4	Inf	Null	
5	Inf	Null	
6	Inf	Null	
7	Inf	Null	
8	Inf	Null	
9	Inf	Null	

Pop 2



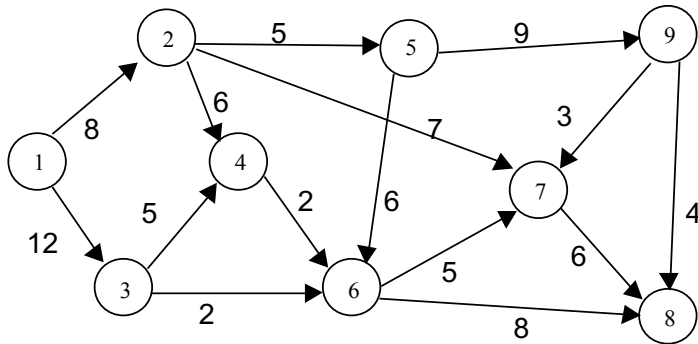
	D[]	pred[]	Q[]
1	0	Null	
2	8	1	
3	12	1	1
4	14	2	2
5	13	2	3
6	Inf	Null	
7	15	2	4
8	Inf	Null	
9	Inf	Null	

Pop 3



	D[]	pred[]	Q[]
1	0	Null	
2	8	1	
3	12	1	
4	14	2	1
5	13	2	2
6	14	3	4
7	15	2	3
8	Inf	Null	
9	Inf	Null	

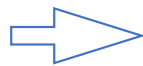
Example - 3



If $D[v] > D[u] + w(u, v)$, set $D[v] = D[u] + w(u, v)$

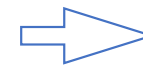
	D[]	pred[]	Q[]
1	0	Null	
2	8	1	
3	12	1	
4	14	2	1
5	13	2	2
6	14	3	4
7	15	2	3
8	Inf	Null	
9	Inf	Null	

Pop 4



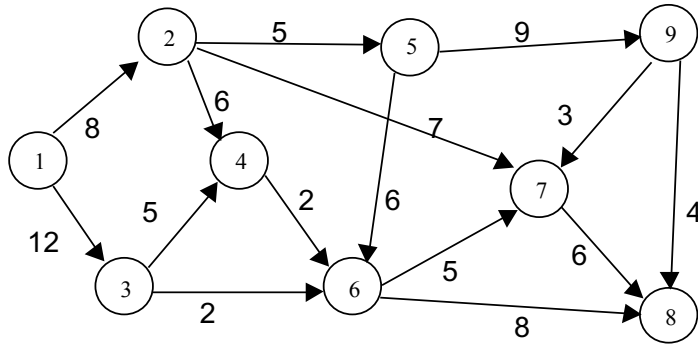
	D[]	pred[]	Q[]
1	0	Null	
2	8	1	
3	12	1	
4	14	2	
5	13	2	1
6	14	3	3
7	15	2	2
8	Inf	Null	
9	Inf	Null	

Pop 5



	D[]	pred[]	Q[]
1	0	Null	
2	8	1	
3	12	1	
4	14	2	
5	13	2	
6	14	3	2
7	15	2	1
8	Inf	Null	
9	22	5	3

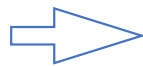
Example - 4



If $D[v] > D[u] + w(u, v)$, set $D[v] = D[u] + w(u, v)$

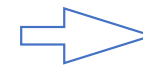
	D[]	pred[]	Q[]
1	0	Null	
2	8	1	
3	12	1	
4	14	2	
5	13	2	
6	14	3	2
7	15	2	1
8	Inf	Null	
9	22	5	3

Pop 7



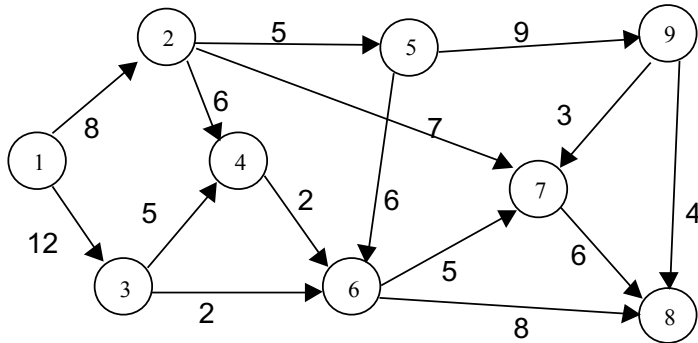
	D[]	pred[]	Q[]
1	0	Null	
2	8	1	
3	12	1	
4	14	2	
5	13	2	
6	14	3	1
7	15	2	
8	21	7	3
9	22	5	2

Pop 6



	D[]	pred[]	Q[]
1	0	Null	
2	8	1	
3	12	1	
4	14	2	
5	13	2	
6	14	3	
7	15	2	
8	21	7	2
9	22	5	1

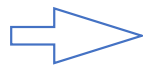
Example - 5



If $D[v] > D[u] + w(u, v)$, set $D[v] = D[u] + w(u, v)$

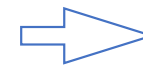
	D[]	pred[]	Q[]
1	0	Null	
2	8	1	
3	12	1	
4	14	2	
5	13	2	
6	14	3	
7	15	2	
8	21	7	2
9	22	5	1

Pop 9



	D[]	pred[]	Q[]
1	0	Null	
2	8	1	
3	12	1	
4	14	2	
5	13	2	
6	14	3	
7	15	2	
8	21	7	1
9	22	5	

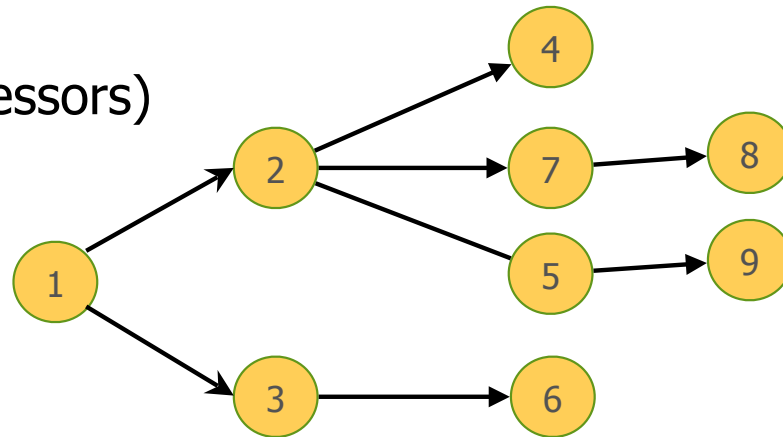
Pop 8



	D[]	pred[]	Q[]
1	0	Null	
2	8	1	
3	12	1	
4	14	2	
5	13	2	
6	14	3	
7	15	2	
8	21	7	
9	22	5	

Example - 6

- Observations
 - Each vertex left queue only once, so edges out of each vertex were scanned only once
 - With simple data structure, this is much faster than Dijkstra
 - Compared with full Bellman-Ford, each edge would be scanned 8 times ($|V| - 1$)
- Final shortest path tree (use predecessors)



Bellman-Ford vs Dijkstra

- Dijkstra's algorithm
 - At each step, finds the next vertices that is closest to the start vertex
 - Grows the shortest path tree in order of shortest distance
 - Greedy: no backtracking
 - $O(|E| + |V| \log(|V|))$ with complex data structures
 - Requires non-negative weights
- Bellman-Ford
 - At each step, finds shortest distance to vertices over paths with less than k edges
 - Grows the shortest path tree in order of minimum number of edges from start vertex
 - Backtracks, can scan the same vertex multiple times
 - $O(|E||V|)$ worst case with very simple data structures
 - Can handle negative weights, but no negative cycles
 - Will work asynchronously, perfect for parallel applications

EC504 ALGORITHMS AND DATA STRUCTURES
SPRING 2025 MONDAY & WEDNESDAY
2:30 PM - 4:15 PM

Prof: David Castañón, dac@bu.edu

Teaching Assts:

Seyda Guzelhan : seyda@bu.edu

New Variation: All Pairs Shortest Paths

Input: Directed graph $G = (V, E)$, where $V = \{1, 2, \dots, n\}$, with edge-weight function $w : E \rightarrow \mathbb{R}$

- Weights may be negative, but no negative cycles

Output: $n \times n$ matrix of shortest-path lengths $D(i, j)$ for all $i, j \in V$ (and routes)

Possible solution:

- Run Bellman-Ford once from each vertex as starting node
- Time $O(|V|^2 E)$
- Dense graph ($\Theta(n^2)$ edges) $\Rightarrow O(n^4)$ time in the worst case

New Variation: All Pairs Shortest Paths

Input: Directed graph $G = (V, E)$, where $V = \{1, 2, \dots, n\}$, with edge-weight function $w : E \rightarrow \mathbb{R}$

- Assume all weights are non-negative

Possible solution:

- Run Dijkstra once from each vertex as starting node
- Time $O(|V||E| + |V|^2 \log(|V|))$
- Dense graph ($\Theta(n^2)$) $\Rightarrow O(n^3)$ time in the worst case with array priority queue

Better Approach: Use Dynamic Programming

- Concept

- Let $D(i,j)$ be the shortest distance from vertex i to vertex j .
- Assume $D(j,j) = 0$
- Then, shortest path from i to j must have at least one vertex before j (could be i)

$$D(i, j) = \min_{k: (k,j) \in E} [D[i, k] + w(k, j)]$$

- Bellman's principle of optimality: shortest path is composed of shortest paths

$$D(i, j) = \min_{k: k \in V} [D[i, k] + D[k, j]]$$

Floyd-Warshall Algorithm

- Initialize $n \times n$ weighted adjacency matrix A and initial distance matrix $D^{(0)}$ as

$$A(i, j) = \begin{cases} 0 & \text{if } i = j \\ w(i, j) & \text{if } (i, j) \in E, \\ \infty & \text{if } (i, j) \notin E \end{cases} \quad D^{(0)}(i, j) = \begin{cases} 0 & \text{if } i = j \\ \infty & \text{if } i \neq j \end{cases}$$

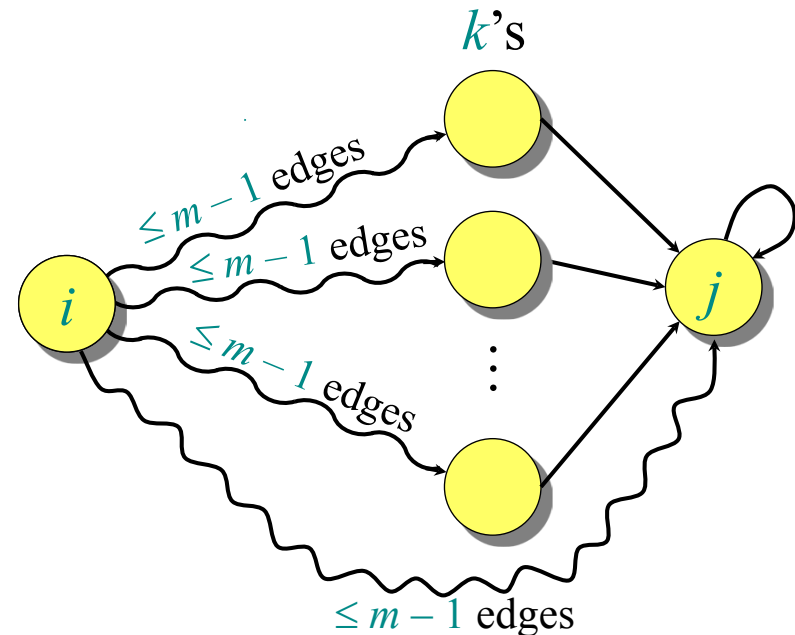
- Note that $D^{(0)}$ is the matrix of shortest distances among paths with zero edges!
- A simple algorithm
 - For $m = 1, 2, \dots, n-1$:

$$D^{(m)}(i, j) = \min_{k \in V} [D^{(m-1)}(i, k) + A(k, j)]$$

- Note that $A(j, j) = 0$, so that $D^{(m)}(i, j) = D^{(m-1)}(i, j)$ is possible, and thus $D^{(m)}(i, j)$ is non-increasing

Floyd-Warshall Algorithm

- Claim: $D^{(m)}(i, j)$ is shortest distance from i to j among paths with m or less edges
 - Any path with less than or equal to m edges must have next to last vertex with shortest path less than $m-1$ edges
 - Principle of optimality



Floyd-Warshall Algorithm

- Variation: faster algorithm with less memory

$$\text{Initially } D(i, j) = \begin{cases} w(i, j) & \text{if } (i, j) \in E \\ \infty & \text{if } (i, j) \notin E \end{cases}$$

- For k in V :

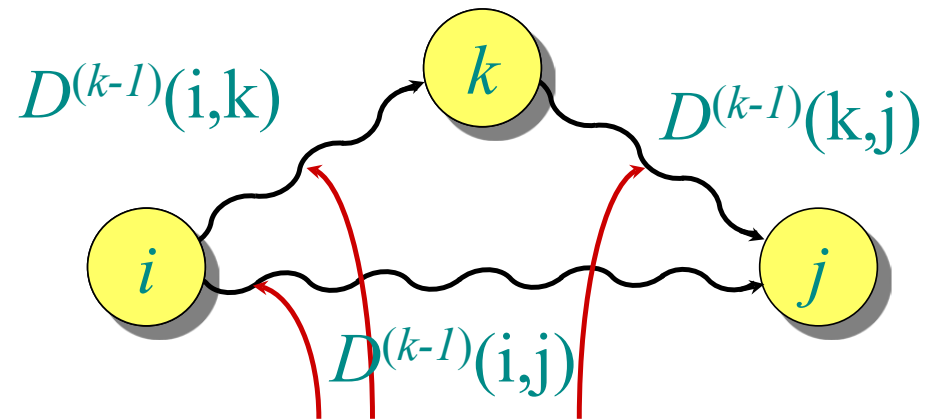
- For i in V :

- For j in V :

If $D(i, j) > D(i, k) + D(k, j)$:

set $D(i, j) = D(i, k) + D(k, j)$

- Complexity: $O(|V|^3)$



Floyd-Warshall Algorithm

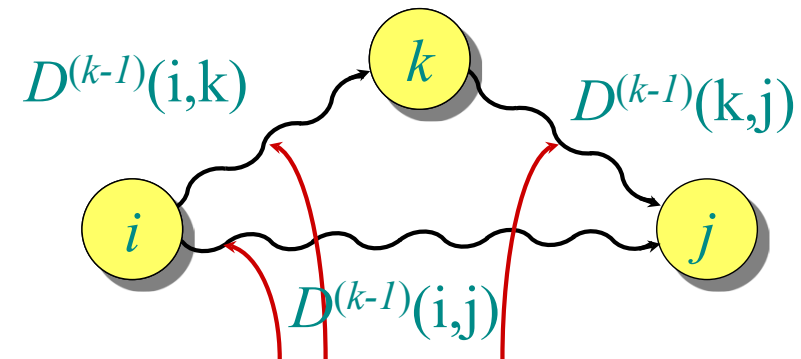
- Keeping track of paths

$$\text{Initially } \text{pred}(i, j) = \begin{cases} i & (i, j) \in E \\ -1 & (i, j) \notin E \end{cases} \quad D(i, j) = \begin{cases} w(i, j) & \text{if } (i, j) \in E \\ \infty & \text{if } (i, j) \notin E \end{cases}$$

- For k in V :
 - For i in V :
 - For j in V :

If $D(i, j) > D(i, k) + D(k, j)$:

set $D(i, j) = D(i, k) + D(k, j)$, $\text{pred}(i, j) = \text{pred}(k, j)$



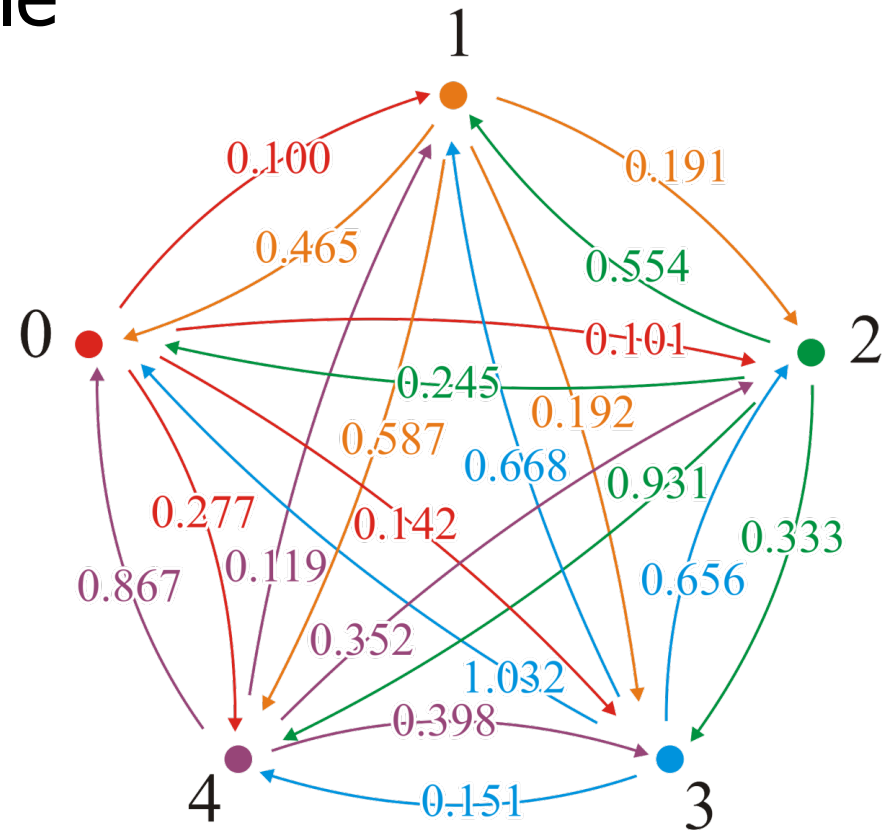
Example

- Weighted adjacency matrix

▸ $A =$

0	0.1	0.101	0.142	0.277
0.465	0	0.191	0.192	0.587
0.245	0.554	0	0.333	0.931
1.032	0.668	0.656	0	0.151
0.867	0.119	0.352	0.398	0

Initially, $D = A$ (fully connected)



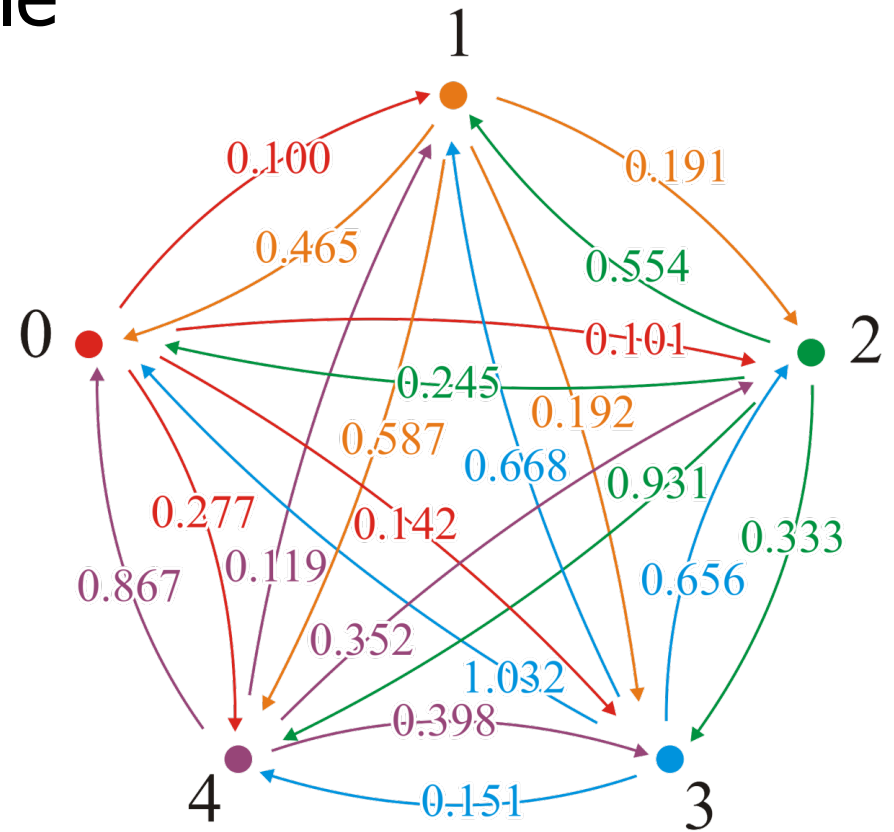
Example

- Iteration for $k = 0$

▸ $D =$

0	0.1	0.101	0.142	0.277
0.465	0	0.191	0.192	0.587
0.245	0.345	0	0.333	0.522
1.032	0.668	0.656	0	0.151
0.867	0.119	0.352	0.398	0

-1	0	0	0	0
1	-1	1	1	1
2	0	-1	2	0
3	3	3	-1	3
4	4	4	4	-1



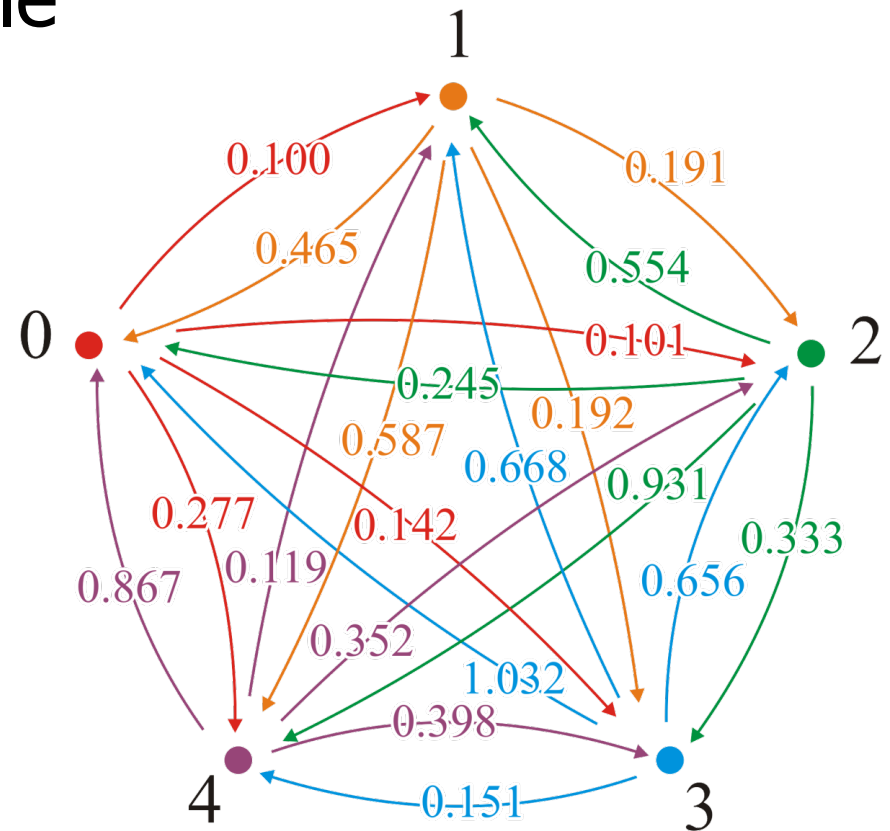
Example

- Iteration for $k = 1$

▸ $D =$

0	0.1	0.101	0.142	0.277
0.465	0	0.191	0.192	0.587
0.245	0.345	0	0.333	0.522
1.032	0.668	0.656	0	0.151
0.584	0.119	0.31	0.311	0

-1	0	0	0	0
1	-1	1	1	1
2	0	-1	2	0
3	3	3	-1	3
1	4	1	1	-1



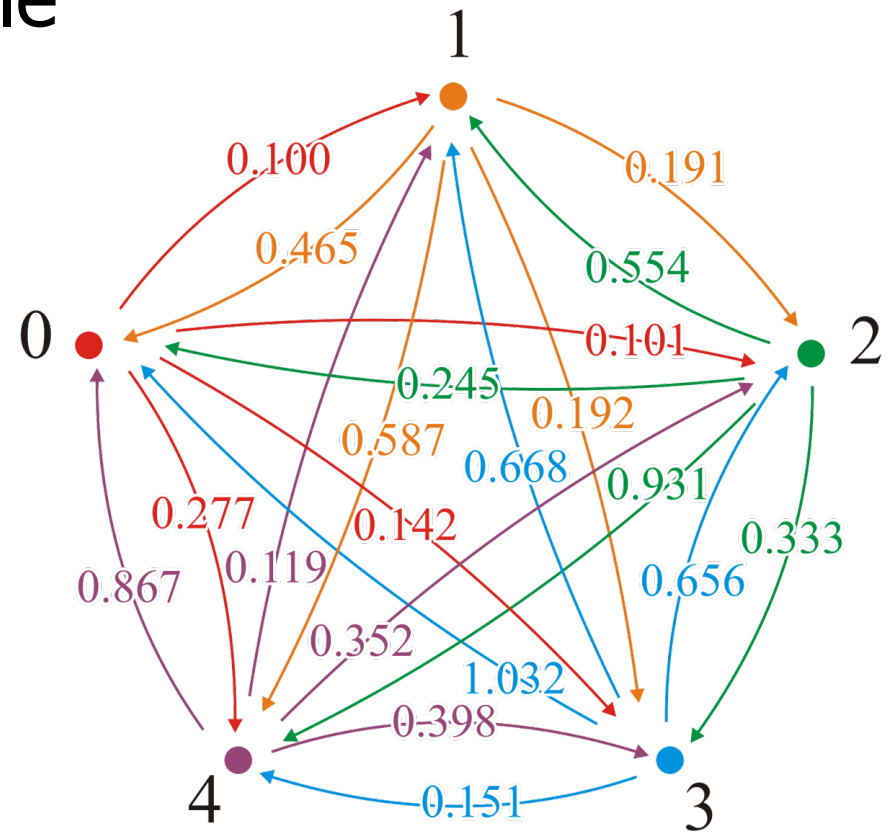
Example

Iteration for $k = 2$

D=

0	0.1	0.101	0.142	0.277
0.436	0	0.191	0.192	0.587
0.245	0.345	0	0.333	0.522
0.901	0.668	0.656	0	0.151
0.555	0.119	0.31	0.311	0

-1	0	0	0	0
2	-1	1	1	1
2	0	-1	2	0
2	3	3	-1	3
2	4	1	1	-1



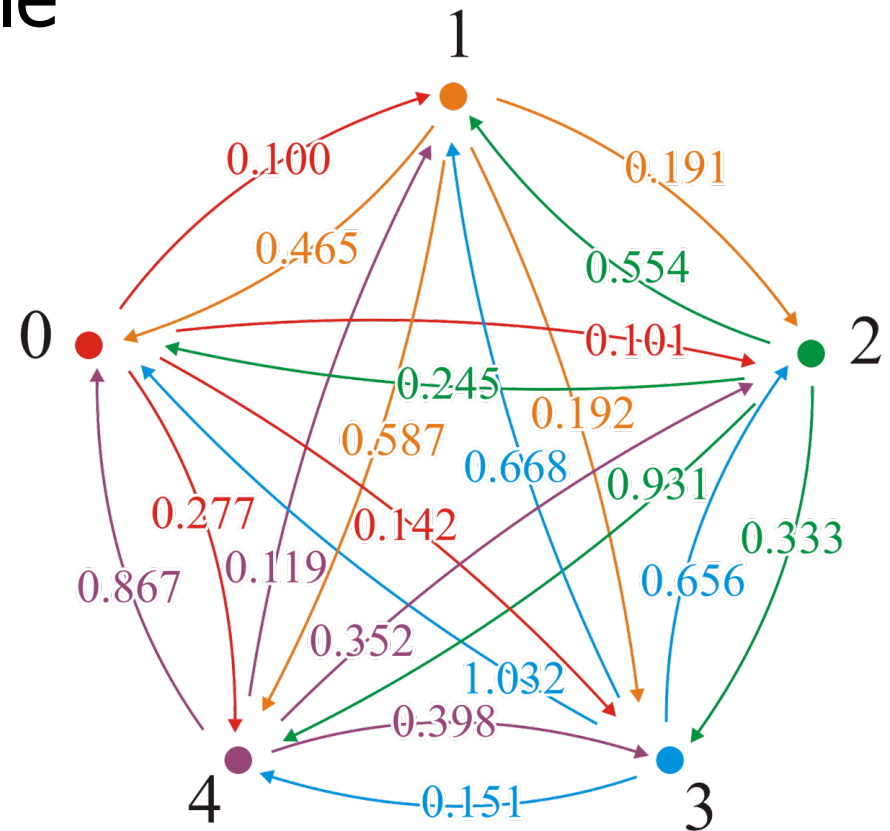
Example

Iteration for $k = 3$

D=

0	0.1	0.101	0.142	0.277
0.436	0	0.191	0.192	0.343
0.245	0.345	0	0.333	0.484
0.901	0.668	0.656	0	0.151
0.555	0.119	0.31	0.311	0

-1	0	0	0	0
2	-1	1	1	3
2	0	-1	2	3
2	3	3	-1	3
2	4	1	1	-1



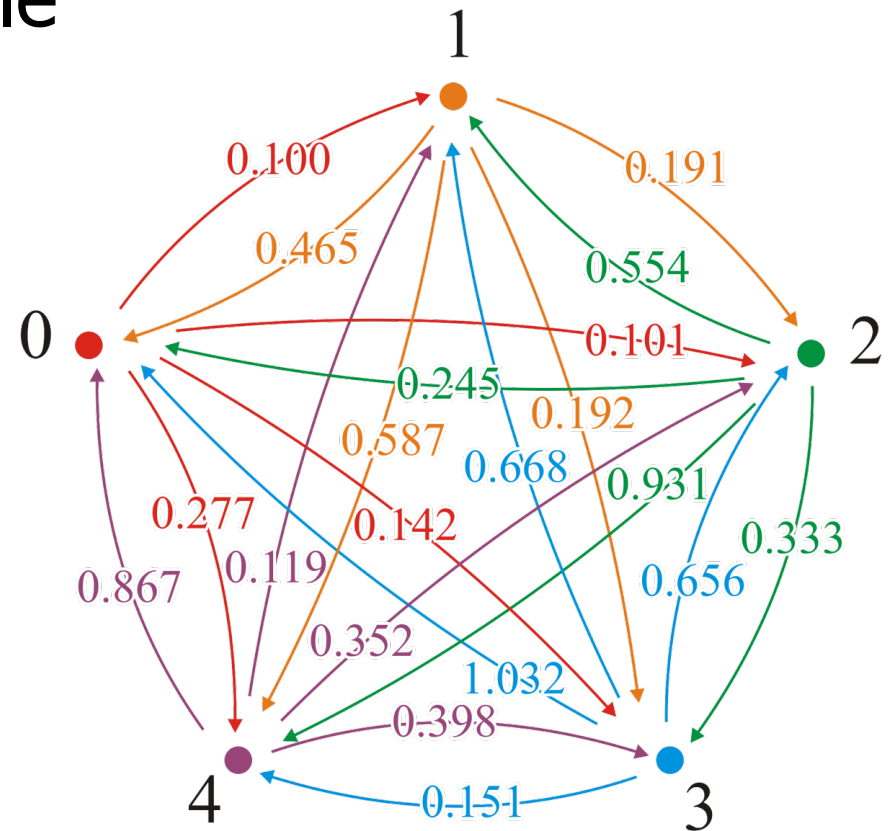
Example

- Iteration for $k = 4$

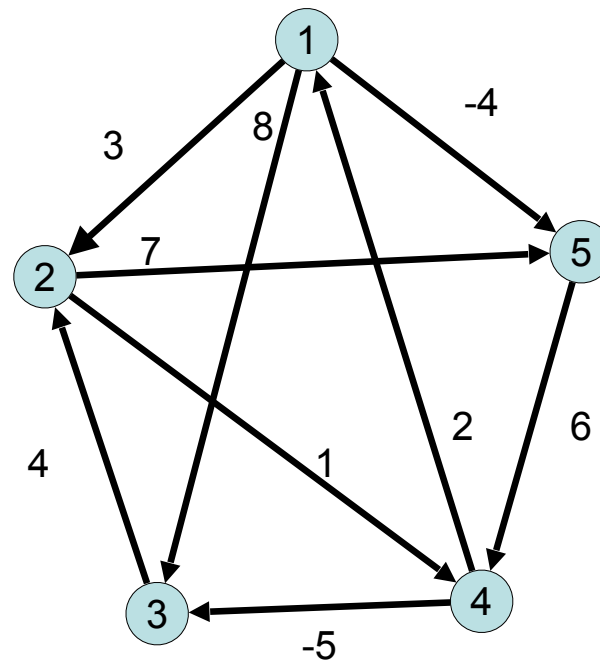
▸ $D =$

0	0.1	0.101	0.142	0.277
0.436	0	0.191	0.192	0.343
0.245	0.345	0	0.333	0.484
0.706	0.27	0.461	0	0.151
0.555	0.119	0.31	0.311	0

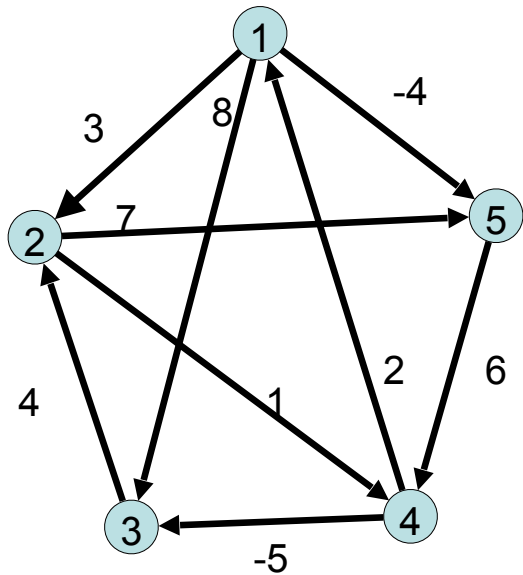
-1	0	0	0	0
2	-1	1	1	3
2	0	-1	2	3
2	4	1	-1	3
2	4	1	1	-1



Floyd-Warshall: Example 2



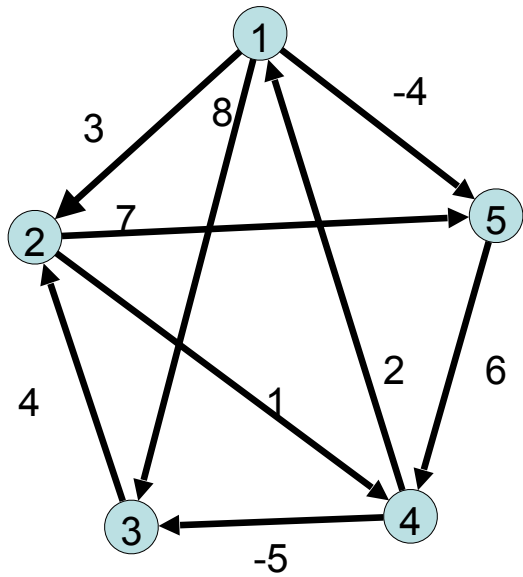
All-Pairs Shortest Paths



Floyd-Warshall

Initial distances and predecessors

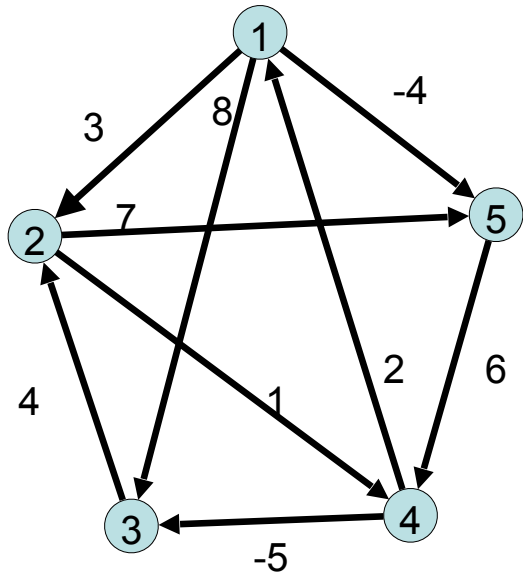
$$D^{(0)} = \begin{pmatrix} 0 & 3 & 8 & \infty & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & \infty & \infty \\ 2 & \infty & -5 & 0 & \infty \\ \infty & \infty & \infty & 6 & 0 \end{pmatrix} \quad \Pi^{(0)} = \begin{pmatrix} NIL & 1 & 1 & NIL & 1 \\ NIL & NIL & NIL & 2 & 2 \\ NIL & 3 & NIL & NIL & NIL \\ 4 & NIL & 4 & NIL & NIL \\ NIL & NIL & NIL & 5 & NIL \end{pmatrix}$$



Floyd-Warshall

After k = 1

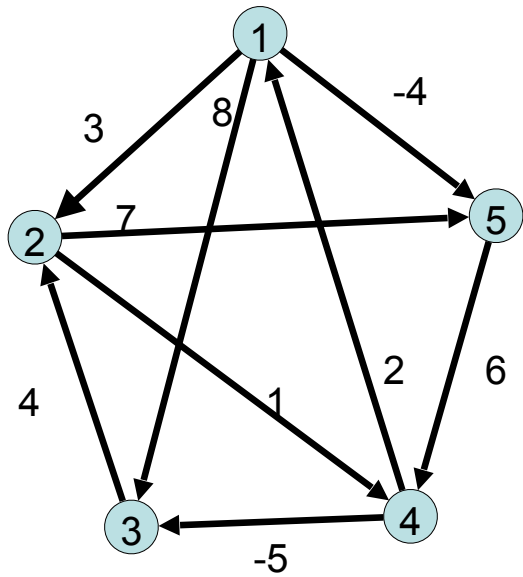
$$D^{(1)} = \begin{pmatrix} 0 & 3 & 8 & \infty & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & \infty & \infty \\ 2 & 5 & -5 & 0 & -2 \\ \infty & \infty & \infty & 6 & 0 \end{pmatrix} \quad \Pi^{(1)} = \begin{pmatrix} NIL & 1 & 1 & NIL & 1 \\ NIL & NIL & NIL & 2 & 2 \\ NIL & 3 & NIL & NIL & NIL \\ 4 & 1 & 4 & NIL & 1 \\ NIL & NIL & NIL & 5 & NIL \end{pmatrix}$$



Floyd-Warshall

After k = 2

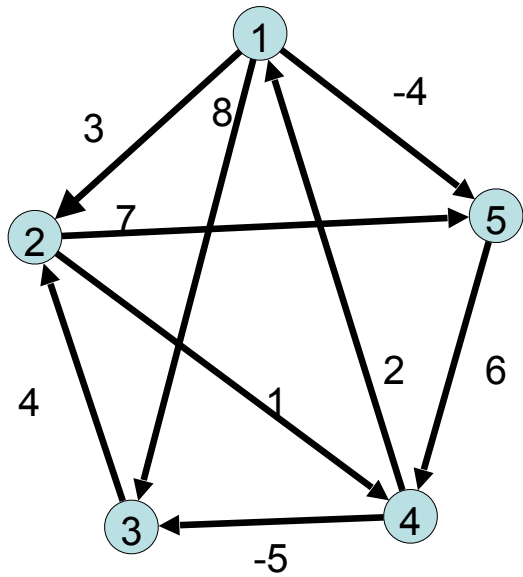
$$D^{(2)} = \begin{pmatrix} 0 & 3 & 8 & 4 & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & 5 & 11 \\ 2 & 5 & -5 & 0 & -2 \\ \infty & \infty & \infty & 6 & 0 \end{pmatrix} \quad \Pi^{(2)} = \begin{pmatrix} NIL & 1 & 1 & 2 & 1 \\ NIL & NIL & NIL & 2 & 2 \\ NIL & 3 & NIL & 2 & 2 \\ 4 & 1 & 4 & NIL & 1 \\ NIL & NIL & NIL & 5 & NIL \end{pmatrix}$$



Floyd-Warshall

After k = 3

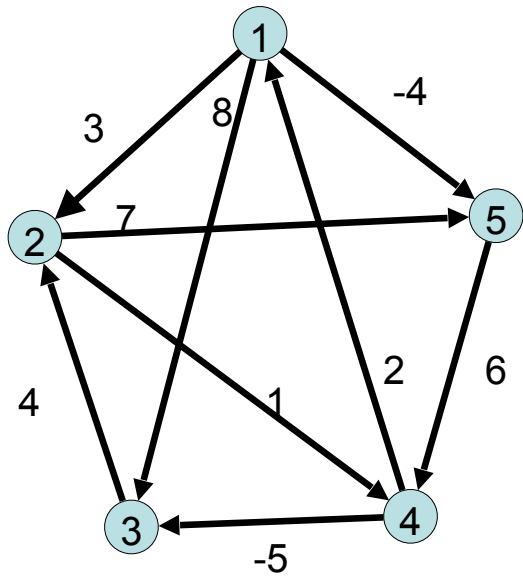
$$D^{(3)} = \begin{pmatrix} 0 & 3 & 8 & 4 & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & 5 & 11 \\ 2 & -1 & -5 & 0 & -2 \\ \infty & \infty & \infty & 6 & 0 \end{pmatrix} \quad \Pi^{(3)} = \begin{pmatrix} NIL & 1 & 1 & 2 & 1 \\ NIL & NIL & NIL & 2 & 2 \\ NIL & 3 & NIL & 2 & 2 \\ 4 & 3 & 4 & NIL & 1 \\ NIL & NIL & NIL & 5 & NIL \end{pmatrix}$$



Floyd-Warshall

After k = 4

$$D^{(4)} = \begin{pmatrix} 0 & 3 & -1 & 4 & -4 \\ 3 & 0 & -4 & 1 & -1 \\ 7 & 4 & 0 & 5 & 3 \\ 2 & -1 & -5 & 0 & -2 \\ 8 & 5 & 1 & 6 & 0 \end{pmatrix} \quad \Pi^{(4)} = \begin{pmatrix} NIL & 1 & 4 & 2 & 1 \\ 4 & NIL & 4 & 2 & 1 \\ 4 & 3 & NIL & 2 & 1 \\ 4 & 3 & 4 & NIL & 1 \\ 4 & 3 & 4 & 5 & NIL \end{pmatrix}$$



Floyd-Warshall

After k = 5 (final)

$$D^{(5)} = \begin{pmatrix} 0 & 1 & -3 & 2 & -4 \\ 3 & 0 & -4 & 1 & -1 \\ 7 & 4 & 0 & 5 & 3 \\ 2 & -1 & -5 & 0 & -2 \\ 8 & 5 & 1 & 6 & 0 \end{pmatrix} \quad \Pi^{(5)} = \begin{pmatrix} NIL & 3 & 4 & 5 & 1 \\ 4 & NIL & 4 & 2 & 1 \\ 4 & 3 & NIL & 2 & 1 \\ 4 & 3 & 4 & NIL & 1 \\ 4 & 3 & 4 & 5 & NIL \end{pmatrix}$$

Can We Do Better for Sparse Graphs?

- If graph is sparse, we can do better with Dijkstra from every vertex, provided weights are non-negative
 - Dijkstra: $O(|V||E| + |V|^2 \log(|V|))$ vs Floyd-Warshall $O(|V|^3)$
- Can we apply Dijkstra when graph has negative weight edges, but not negative cycles?
 - Graph reweighing

Graph Reweighing

Theorem. Given a function $h : V \rightarrow \mathbb{R}$, **reweigh** each edge $(u,v) \in E$ by $w_h(u,v) = w(u,v) + h(u) - h(v)$.

Then, for any two vertices, all paths between them are reweighed by the same amount
 $\implies$ shortest path in reweighed graph is also shortest in original path

Proof. Let $p = v_1 \rightarrow v_2 \rightarrow \dots \rightarrow v_k$ be a path in the original graph G . We have

$$w(p) = w(v_1, v_2) + w(v_2, v_3) + \dots + w(v_{k-1}, v_k)$$

On the reweighed graph G_h , we have

$$\begin{aligned} w_h(p) &= w_h(v_1, v_2) + w_h(v_2, v_3) + \dots + w_h(v_{k-1}, v_k) \\ &= w(v_1, v_2) + h(v_1) - h(v_2) + w(v_2, v_3) + h(v_2) - h(v_3) + \dots + w(v_{k-1}, v_k) + h(v_{k-1}) - h(v_k) \\ &= w(p) + h(v_1) - h(v_k) \end{aligned}$$

Same amount for every path that starts at v_1 , ends at v_k !

Can We Do Better for Sparse Graphs?

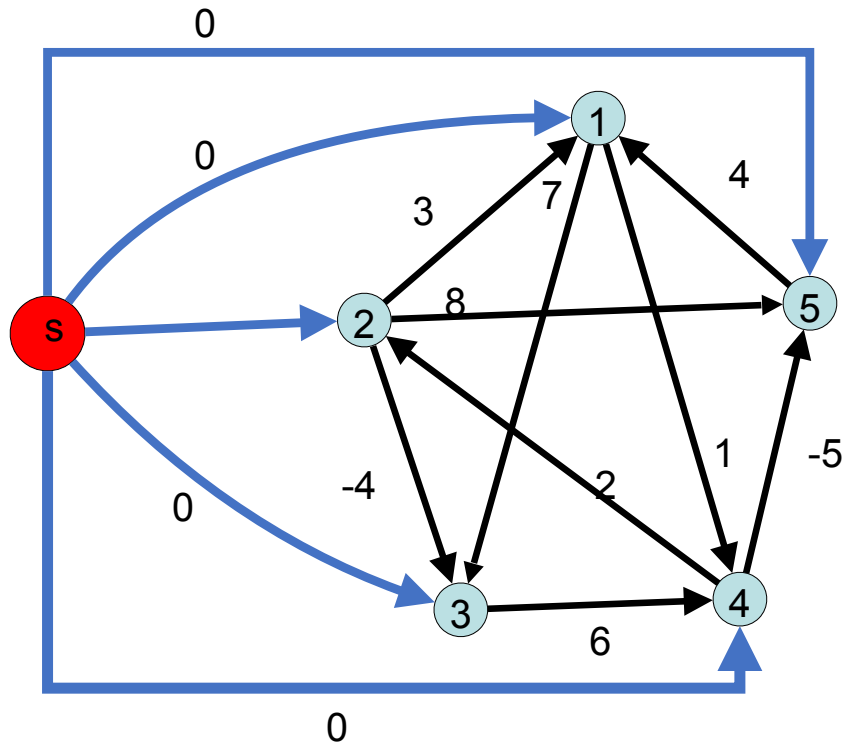
- So, want to find a reweighing function such that $w_h(u, v) \geq 0$ for all edges in E
 - Then, we could solve using Dijkstra's algorithm starting from each vertex
- **Johnson's** algorithm
 - Use an auxiliary shortest path problem solved with Bellman-Ford to using Bellman-Ford to find $h(v)$ such that $h(v) - h(u) \leq w(u, v)$, or determine that a negative-weight cycle exists.
 - Time = $O(|V| |E|)$
 - Resulting problem with $w_h(u, v) = w(u, v) + h(u) - h(v) \geq 0$

Johnson's algorithm

```
Johnson(G)
{  compute  $G'$ , where  $V[G'] = V[G] \cup \{s\}$  and
     $E[G'] = E[G] \cup \{(s, v) : v \in V[G]\}$ 
  if Bellman-Ford( $G', w, s$ ) = False
    then print "There is a neg-weight cycle"
  else for each vertex  $v \in V[G']$ 
    set  $h(v) = \delta(s, v)$  computed by Bellman-Ford algo.
  for each edge  $(u, v) \in E[G']$ :
     $w'(u, v) = w(u, v) + h(u) - h(v)$ 
  for each vertex  $u \in V[G']$ 
    run Dijkstra( $G, w', u$ ) to compute  $\delta'(u, v)$ 
  for each  $v \in V[G]$ 
     $d_{uv} = \delta'(u, v) - h(u) + h(v)$ 

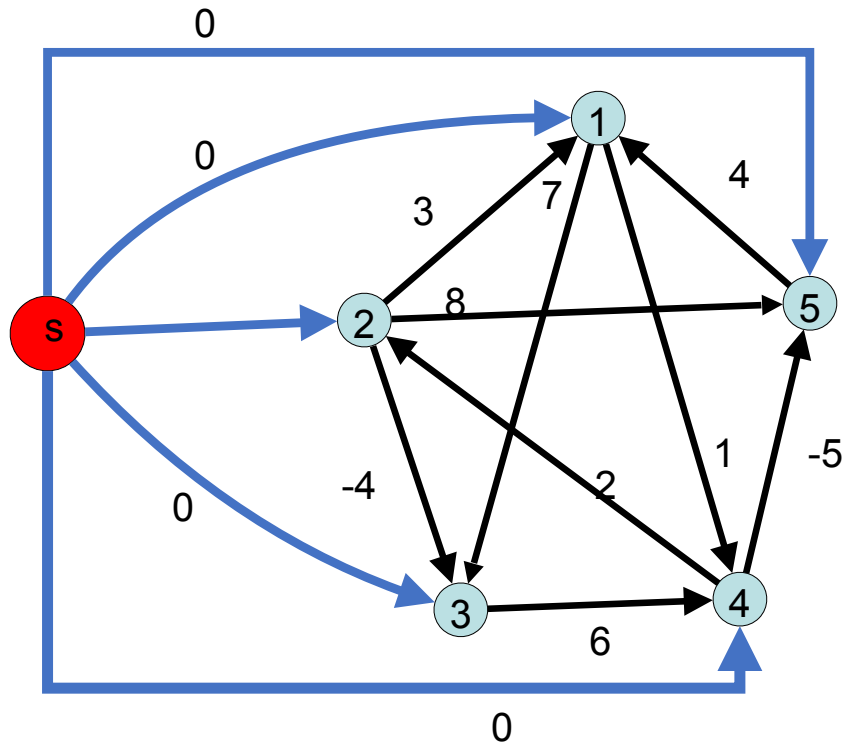
  return D
}
```

Johnson's algorithm



Main idea: Add a dummy vertex with a directed edge of weight 0 to all other vertices

Johnson's algorithm

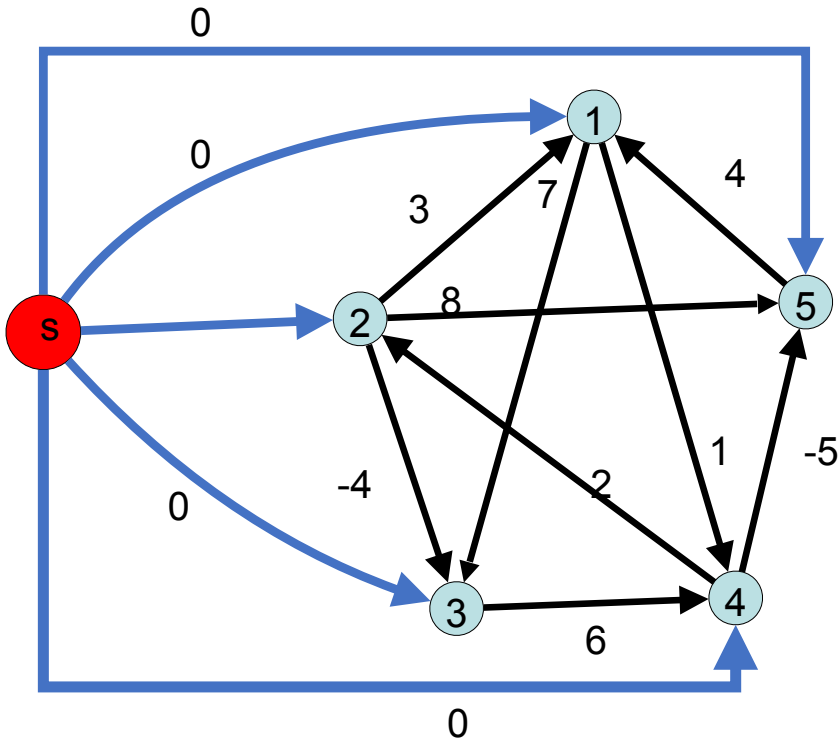


Use Bellman-Ford from s to get shortest distance to all other nodes:

Must be negative

	D[]	pred[]	Q[]
1	0	S	1
2	0	S	2
3	0	S	3
4	0	S	4
5	0	S	5

Johnson's algorithm



All-Pairs Shortest Paths

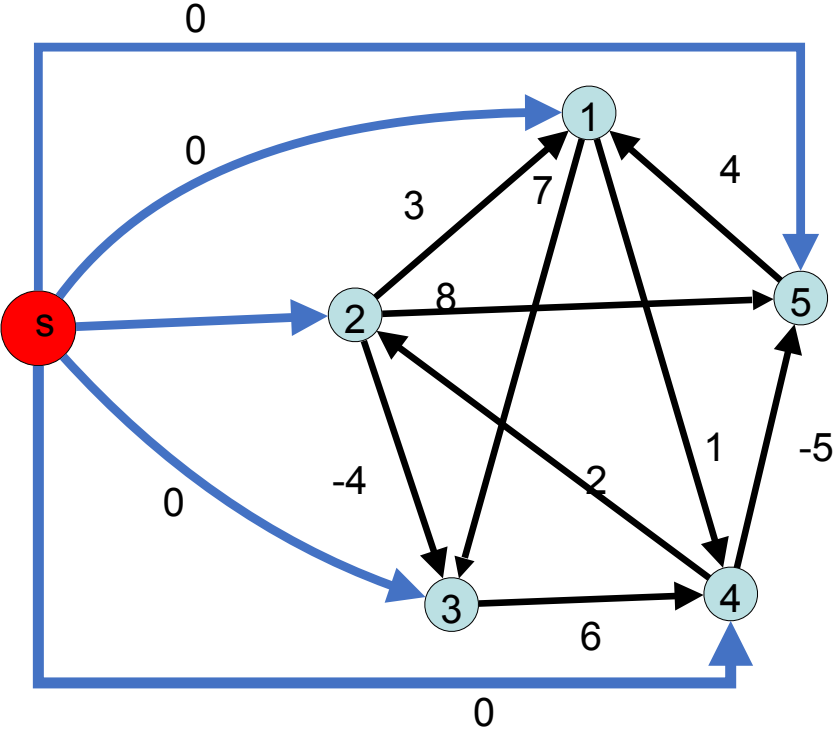
Pop 1

	D[]	pred[]	Q[]
1	0	S	
2	0	S	1
3	0	S	2
4	0	S	3
5	0	S	4

Pop 2

	D[]	pred[]	Q[]
1	0	S	
2	0	S	
3	-4	2	1
4	0	S	2
5	0	S	3

Johnson's algorithm



All-Pairs Shortest Paths

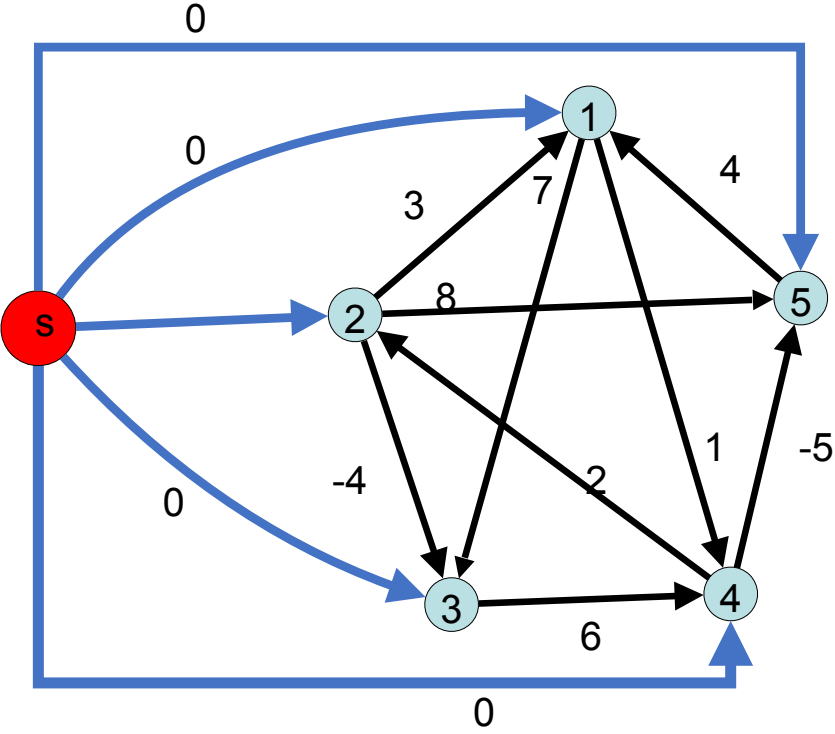
Pop 3

	D[]	pred[]	Q[]
1	0	S	
2	0	S	
3	-4	2	
4	0	S	1
5	0	S	2

Pop 4

	D[]	pred[]	Q[]
1	0	S	
2	0	S	
3	-4	2	
4	0	S	
5	-5	4	1

Johnson's algorithm



All-Pairs Shortest Paths

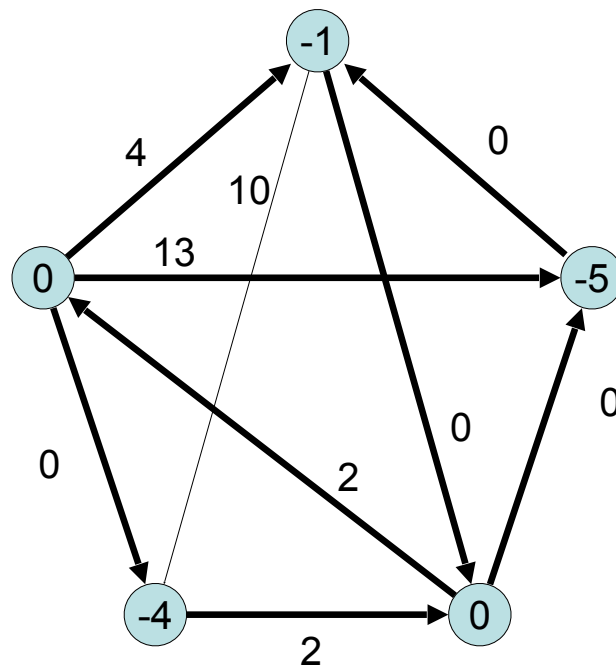
Pop 5

	D[]	pred[]	Q[]
1	-1	5	1
2	0	S	
3	-4	2	
4	0	S	
5	-5	4	

Pop 1

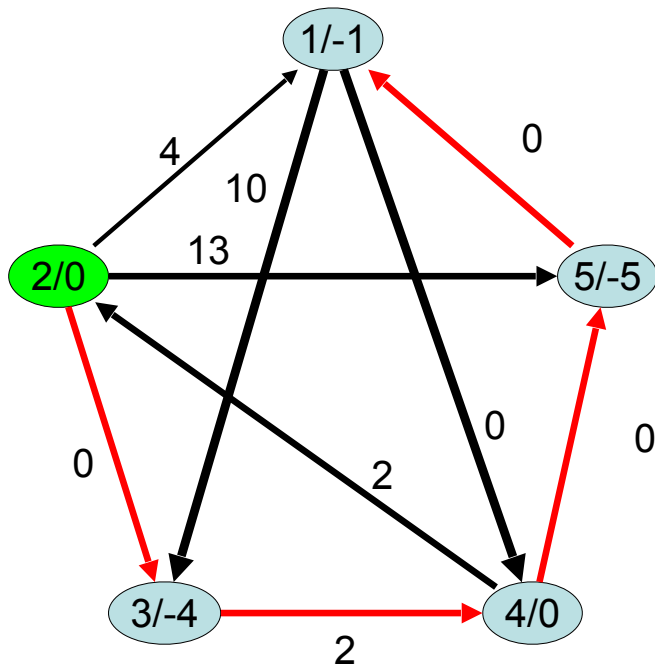
	D[]	pred[]	Q[]
1	-1	5	
2	0	S	
3	-4	2	
4	0	S	
5	-5	4	

Johnson's algorithm



Use Distance from
previous problem
for reweighing

Johnson's algorithm



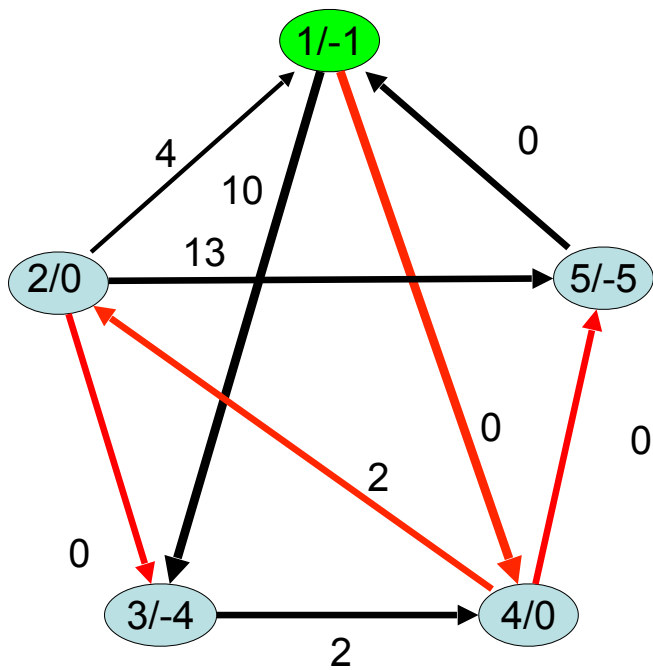
	D[]	pred[]	Q[]
1	Inf	Null	
2	0	null	0
3	Inf	Null	
4	Inf	Null	
5	inf	Null	

	D[]	pred[]	Q[]
1	Inf	Null	
2	0	null	
3	0	2	
4	2	3	0
5	13	2	1

	D[]	pred[]	Q[]
1	Inf	Null	
2	0	null	
3	0	2	
4	2	3	
5	2	4	1

	D[]	pred[]	Q[]
1	Inf	Null	
2	0	null	
3	0	2	0
4	Inf	Null	
5	13	2	1

	D[]	pred[]	Q[]
1	Inf	Null	
2	0	null	
3	0	2	
4	2	3	
5	2	4	1



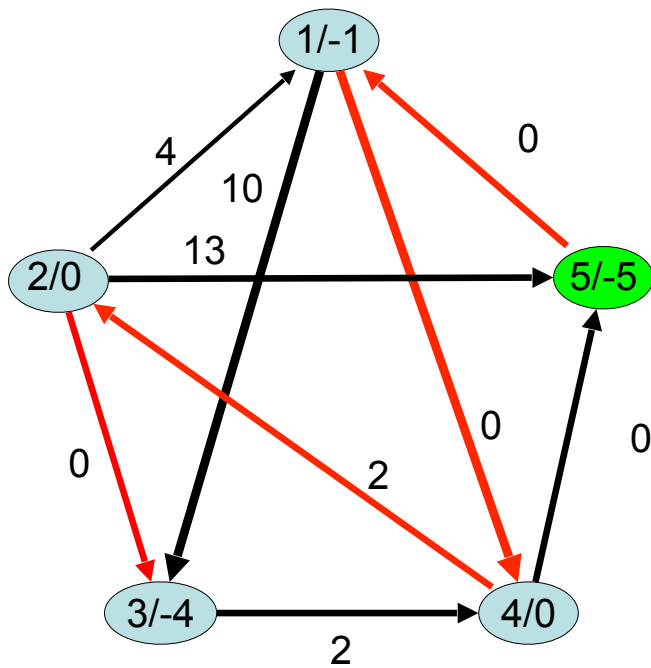
	D[]	pred[]	Q[]
1	0	Null	0
2	Inf	null	
3	Inf	Null	
4	Inf	Null	
5	inf	Null	

	D[]	pred[]	Q[]
1	0	Null	
2	2	null	2
3	10	1	1
4	0	1	
5	0	5	0

	D[]	pred[]	Q[]
1	0	5	
2	2	null	
3	0	2	0
4	0	1	
5	0	5	

	D[]	pred[]	Q[]
1	0	Null	
2	Inf	null	
3	10	1	1
4	0	1	0
5	Inf	null	

	D[]	pred[]	Q[]
1	0	Null	
2	2	null	0
3	10	1	1
4	0	1	
5	0	5	



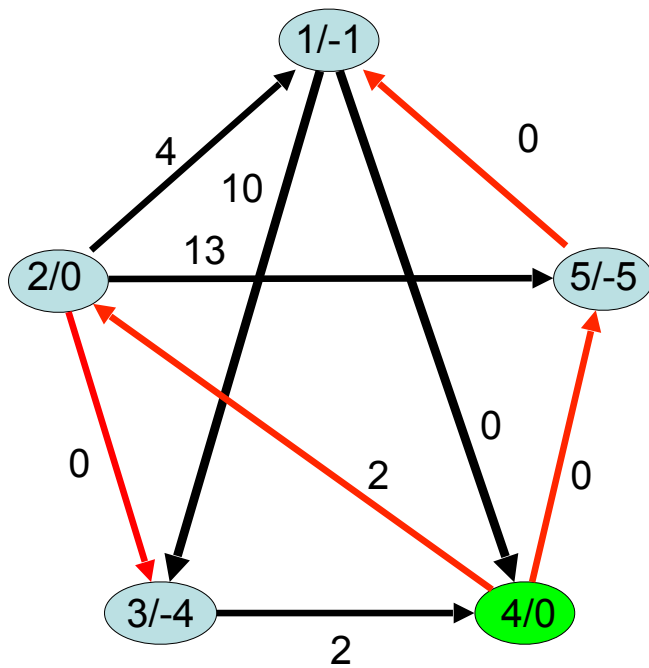
	D[]	pred[]	Q[]
1	Inf	Null	
2	Inf	null	
3	Inf	Null	
4	Inf	Null	
5	0	Null	0

	D[]	pred[]	Q[]
1	0	5	
2	Inf	null	
3	10	1	1
4	0	1	0
5	0	null	

	D[]	pred[]	Q[]
1	0	5	
2	2	4	
3	2	2	0
4	0	1	
5	0	null	

	D[]	pred[]	Q[]
1	0	5	0
2	Inf	null	
3	inf	Null	
4	inf	Null	
5	0	null	

	D[]	pred[]	Q[]
1	0	5	
2	2	4	0
3	10	1	1
4	0	1	
5	0	null	



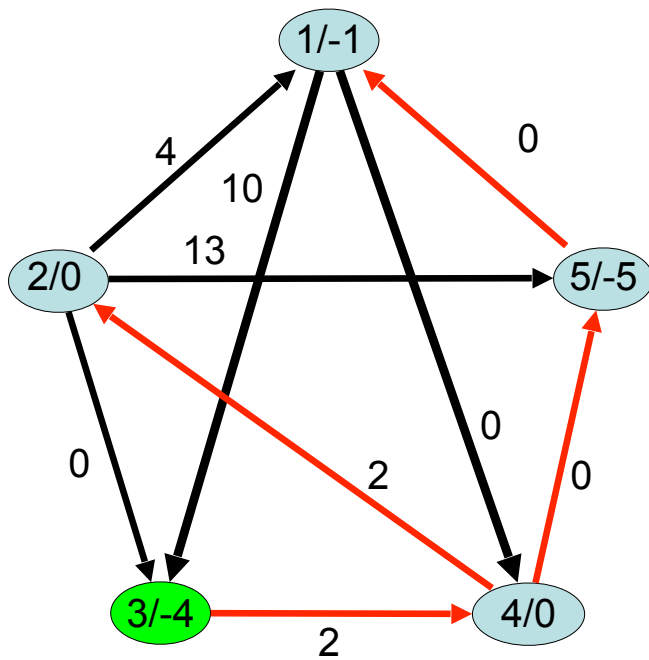
	D[]	pred[]	Q[]
1	Inf	Null	
2	Inf	null	
3	Inf	Null	
4	0	Null	0
5	Inf	Null	

	D[]	pred[]	Q[]
1	0	5	0
2	2	4	1
3	Inf	Null	
4	0	Null	
5	0	4	

	D[]	pred[]	Q[]
1	0	5	
2	2	4	
3	2	3	0
4	0	Null	
5	0	4	

	D[]	pred[]	Q[]
1	Inf	Null	
2	2	4	1
3	Inf	Null	
4	0	Null	
5	0	4	0

	D[]	pred[]	Q[]
1	0	5	
2	2	4	0
3	10	1	1
4	0	Null	
5	0	4	



	D[]	pred[]	Q[]
1	Inf	Null	
2	Inf	null	
3	0	Null	0
4	inf	Null	
5	Inf	Null	

	D[]	pred[]	Q[]
1	Inf	Null	
2	4	4	1
3	0	Null	
4	2	3	
5	2	4	0

	D[]	pred[]	Q[]
1	2	5	
2	4	4	1
3	0	Null	
4	2	3	
5	2	4	

	D[]	pred[]	Q[]
1	Inf	Null	
2	Inf	null	
3	0	Null	
4	2	3	0
5	Inf	Null	

	D[]	pred[]	Q[]
1	2	5	0
2	4	4	1
3	0	Null	
4	2	3	
5	2	4	

EC504 ALGORITHMS AND DATA STRUCTURES
SPRING 2025 MONDAY & WEDNESDAY
2:30 PM - 4:15 PM

Prof: David Castañón, dac@bu.edu

Teaching Assts:

Seyda Guzelhan : seyda@bu.edu

Problem: Shortest Path from s to a single t

- We only want a path to a destination
 - But all algorithms compute paths from s to every destination
 - When can we stop?
 - Can we focus on getting only to where we want to go?
 - This is the Google maps or Waymo problem
- New algorithm: A^* search
 - Will search in direction of t for shortest path
 - A modification of Dijkstra to bias the search
 - Guaranteed optimal under certain conditions

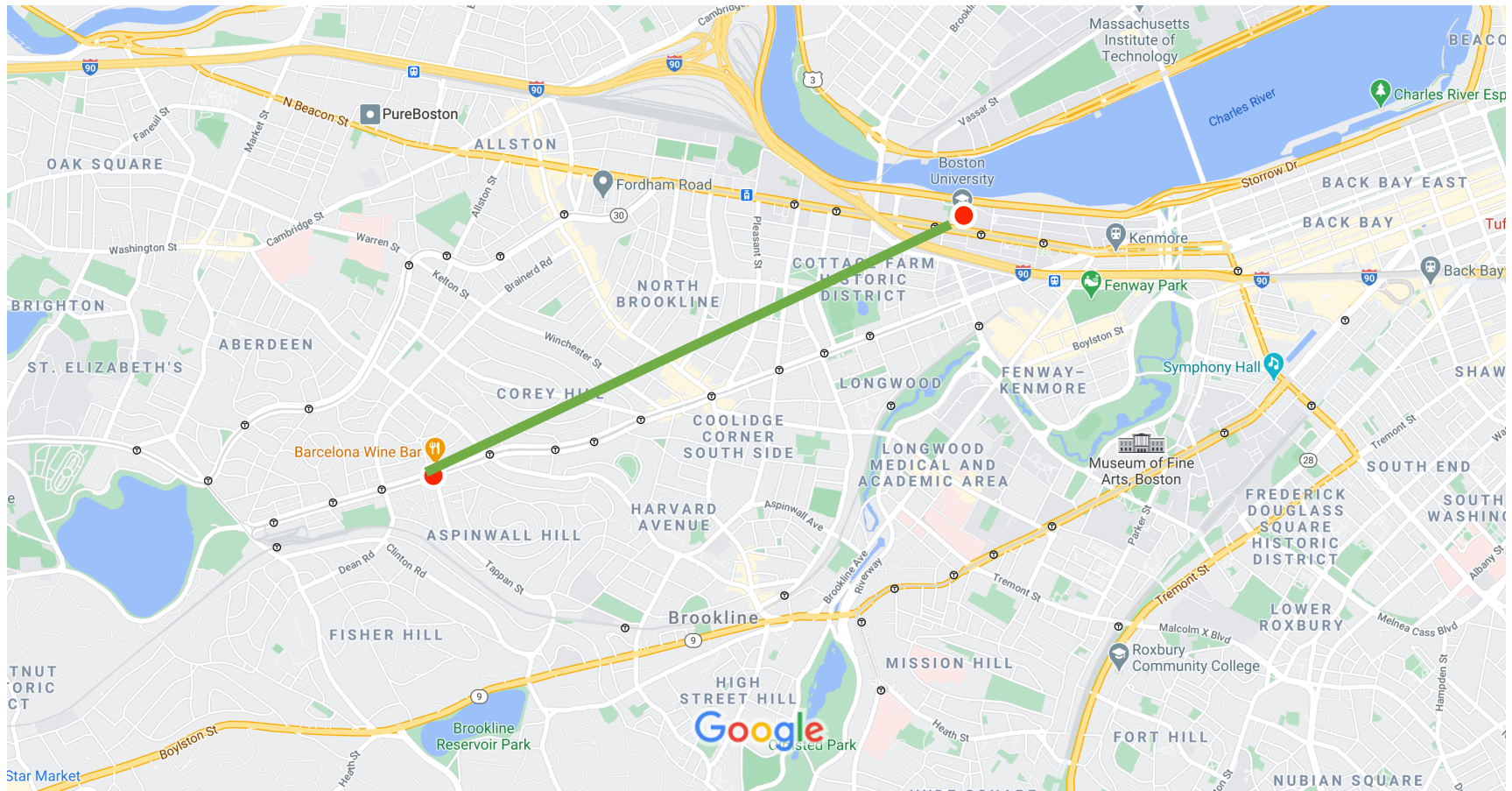
A* search

Example



A* search

Distance lower bound: Straight Path



Heuristic Distance Estimates

- Let $h : V \rightarrow \mathbb{R}$ be a heuristic function that approximates the distance from vertex v to target t
- We call h **admissible** if: for each vertex v : $h(v) \leq D(v,t)$, where $D(v,t)$ is the true shortest path distance
- We call h **consistent** if: For each (u,v) in E : $h(u) \leq w(u,v) + h(v)$.
 - For example, Euclidean distance to target on the plane
 - Euclidean distance is admissible and consistent

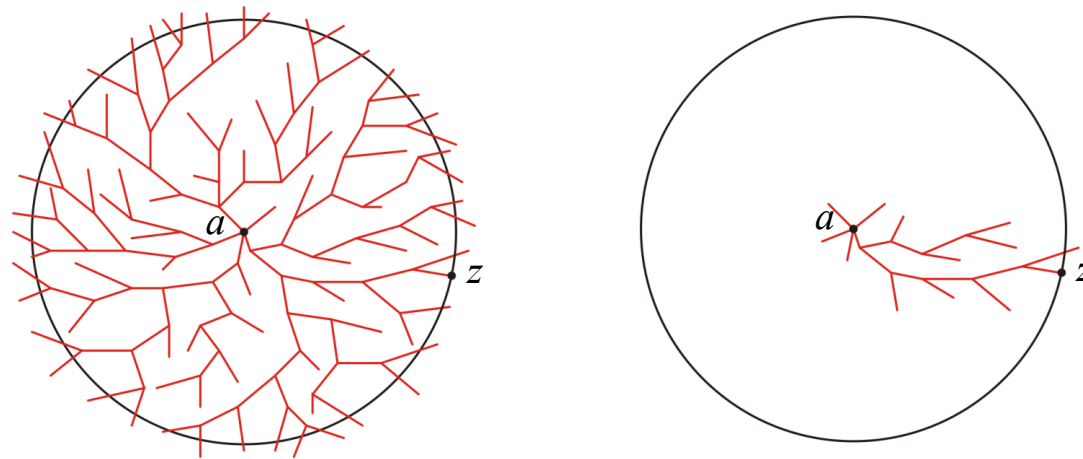
A* Search

- Simple idea: use heuristic $h()$ to reweigh edges
 - $w_h(u, v) = w(u, v) - h(u) + h(v)$
 - Measures deviation from ideal distance estimate by h
 - Consistent heuristics guarantee non-negative weights
- Solve using Dijkstra's algorithm
- Result: If heuristic is consistent and admissible, we get an optimal shortest path
 - Dijkstra's distance is biased towards vertices that have shorter distance estimates

Comparison with Dijkstra's Algorithm

Graphically, we can suggest the behavior of the two algorithms as follows:

- Suppose we are moving from a to z :



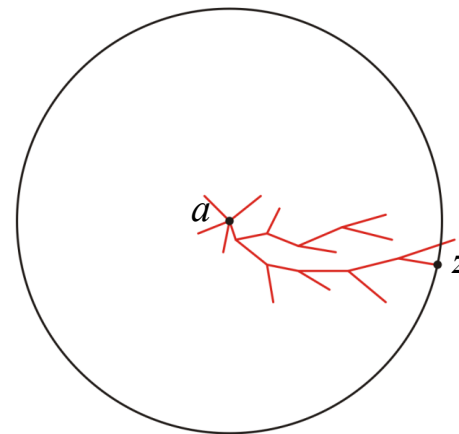
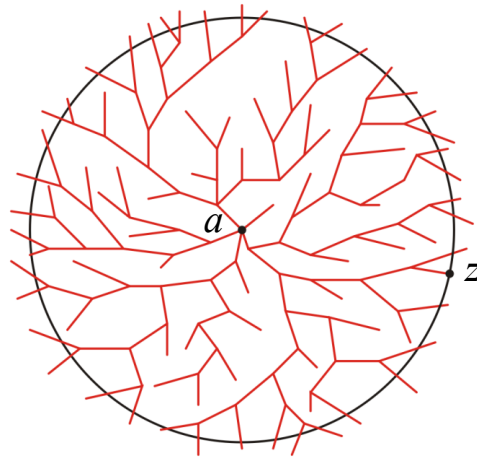
Representative search patterns for Dijkstra's and the A* search algorithms

Comparison with Dijkstra's Algorithm

Dijkstra's algorithm is the A* search algorithm when

using the heuristic distance
$$h(u, v) = \begin{cases} 0 & u = v \\ 1 & u \neq v \end{cases}$$

- No vertex is better than any other vertex



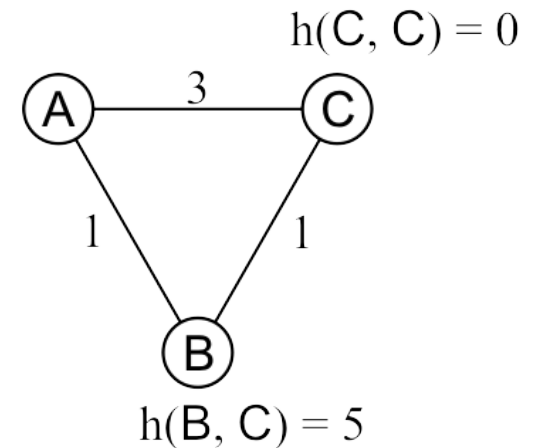
Representative search patterns for Dijkstra's and the A* search algorithms

Algorithm Description

- Keep track of best distance from s , but use as key in the priority queue the optimistic distance estimate to destination t
- Initially, mark distances $D[s] = 0$, $D[v] = \infty$, $v \neq s$, and $\text{key}[v] = D[v] + h(v,t)$, $\text{pred}[v] = \text{NULL}$. Insert into priority queue Q .
- While t is still in Q :
 - Remove_min from Q , as vertex u
 - If $u = t$, break. We have the path we want.
 - For all v such that $(u, v) \in E$, and v is still in Q , compute $D_v = D[u] + w(u, v)$.
 - If $D_v < D[v]$, update $D[v] = D_v$, $\text{pred}[v] = u$, $\text{key}[v] = D_v + h(v,t)$, and Decrease_key of v in Q
- Note: We don't have to keep track of $\text{key}[v]$ explicitly in Q if we know $h(v,t)$

Optimally Guarantees?

- If $h()$ is not consistent or admissible, this algorithm can fail
- Consider simple figure, with $s = A$, $t = C$, and with heuristic $h(B,C) = 5$, $h(C,C) = 0$
- Starting from A,
 - $D[C] = 3$, $\text{key}[C] = 3$, $\text{pred}[C] = A$;
 - $D[B] = 1$, $\text{key}[B] = 6$, $\text{pred}[B] = A$;
- First vertex out of Q is C, so shortest path identified is $A \rightarrow C$
 - Clearly longer than $A \rightarrow B \rightarrow C$
- Problem? Heuristic not admissible (not optimistic)



Problem: Shortest Path from s to a single t

- We only want a path to a destination
 - But all algorithms compute paths from s to every destination
 - When can we stop?
 - Can we focus on getting only to where we want to go?
 - This is the Google maps or Waymo problem
- New algorithm: A^* search
 - Will search in direction of t for shortest path
 - A modification of Dijkstra to bias the search
 - Guaranteed optimal under certain conditions

Heuristic Distance Estimates

- Let $h : V \rightarrow \mathbb{R}$ be a heuristic function that approximates the distance from vertex v to target t
- We call h **admissible** if: for each vertex v : $h(v) \leq D(v,t)$, where $D(v,t)$ is the true shortest path distance
- We call h **consistent** if: For each (u,v) in E : $h(u) \leq w(u,v) + h(v)$.
 - For example, Euclidean distance to target on the plane
 - Euclidean distance is admissible and consistent

Algorithm Description

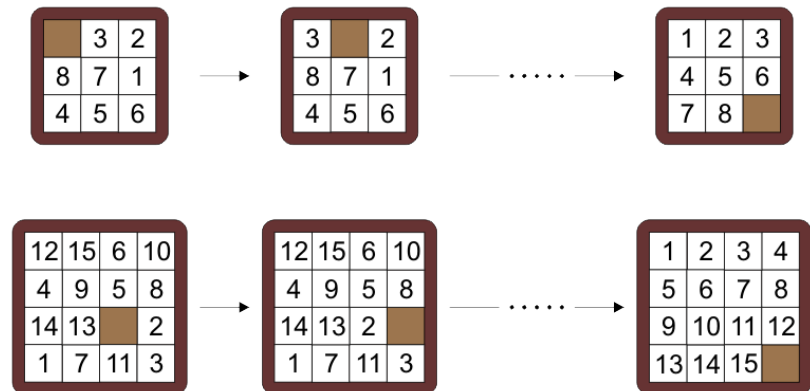
- Keep track of best distance from s , but use as key in the priority queue the optimistic distance estimate to destination t
- Initially, mark distances $D[s] = 0$, $D[v] = \infty$, $v \neq s$, and $\text{key}[v] = D[v] + h(v,t)$, $\text{pred}[v] = \text{NULL}$. Insert into priority queue Q .
- While t is still in Q :
 - Remove_min from Q , as vertex u
 - If $u = t$, break. We have the path we want.
 - For all v such that $(u, v) \in E$, and v is still in Q , compute $D_v = D[u] + w(u, v)$.
 - If $D_v < D[v]$, update $D[v] = D_v$, $\text{pred}[v] = u$, $\text{key}[v] = D_v + h(v,t)$, and Decrease_key of v in Q
- Note: We don't have to keep track of $\text{key}[v]$ explicitly in Q if we know $h(v,t)$

Optimality Results

- **Theorem:** Let $h()$ be admissible and consistent. If A^* selects a path p from s to t , then p is the shortest path
 - Let $w(p)$ be the distance of path p selected by A^* , which is $\text{key}[t]$ when A^* ends
 - Assume for contradiction that another path p' is shorter path to t , so $w(p') < w(p)$
 - Since $h(v, t)$ is admissible, for any vertex v in p' ,
$$w(\{s, v\}) + h(v, t) \leq w(p') < w(p)$$
 - Note, for any vertex v in p' , $\text{key}[v] < w(p)$; furthermore, $\text{key}[t] = w(p)$ when t was removed from the priority queue Q to finish A^*
 - Thus, all vertices v in p' should have been removed from Q before t with $\text{key}[t] = w(p)$
 - This means that t should have been removed from Q with $\text{key}[t] = w(p')$, which is a contradiction that A^* ended with path p

A* Search: Limitations

- Can use lots of memory!
 - Similar to Breadth-First-Search
 - Hence, often replaced by alternatives with bounded-memory
- Example: N-puzzle ($N!$) possible vertices in Priority Queue
 - Need efficient approaches
 - Not all vertices are reachable; only half can be solved... Still large!



A* Search: Example

- Problem: Find shortest path from start *s* to end *g*
 - There is a barrier!
 - Can only move horizontally, vertically as long as there is no barrier
 - Heuristic $h(v,t)$ = sum of vertical and horizontal distance from *v* to *g*
 - Manhattan distance
 - e.g. $h(f,g) = 5$, $h(k,g) = 2$,
 $h(e,g) = 3$

			<i>a</i>	<i>b</i>	
			<i>c</i>	<i>d</i>	<i>e</i>
<i>f</i>	<i>s</i>	<i>h</i>	<i>k</i>	<i>m</i>	<i>n</i>
<i>p</i>	<i>q</i>	<i>r</i>	<i>t</i>	<i>g</i>	

- Execute A* search on this

A* search

A* Search: Example

	D	Key	Pred
a	Inf	Inf	-1
b	Inf	Inf	-1
c	Inf	Inf	-1
d	Inf	Inf	-1
e	Inf	Inf	-1
f	Inf	Inf	-1
g	Inf	Inf	-1
h	Inf	Inf	-1
k	Inf	Inf	-1
m	Inf	Inf	-1
n	Inf	Inf	-1
p	Inf	Inf	-1
q	Inf	Inf	-1
r	Inf	Inf	-1
s	0	4	-1
t	Inf	Inf	-1

	D	Key	Pred
a	Inf	Inf	-1
b	Inf	Inf	-1
c	Inf	Inf	-1
d	Inf	Inf	-1
e	Inf	Inf	-1
f	1	6	S
g	Inf	Inf	-1
h	1	4	S
k	Inf	Inf	-1
m	Inf	Inf	-1
n	Inf	Inf	-1
p	Inf	Inf	-1
q	Inf	Inf	-1
r	Inf	Inf	-1
s	0	4	-1
t	Inf	Inf	-1

			a	b	
			c	d	e
f	s	h	k	m	n
p	q	r	t	g	

A* search

A* Search: Example

	D	Key	Pred
a	Inf	Inf	-1
b	Inf	Inf	-1
c	Inf	Inf	-1
d	Inf	Inf	-1
e	Inf	Inf	-1
f	1	6	S
g	Inf	Inf	-1
h	1	4	S
k	2	4	h
m	Inf	Inf	-1
n	Inf	Inf	-1
p	Inf	Inf	-1
q	Inf	Inf	-1
r	Inf	Inf	-1
s	0	4	-1
t	Inf	Inf	-1

	D	Key	Pred
a	Inf	Inf	-1
b	Inf	Inf	-1
c	3	6	k
d	Inf	Inf	-1
e	Inf	Inf	-1
f	1	6	S
g	Inf	Inf	-1
h	1	4	S
k	2	4	h
m	Inf	Inf	-1
n	Inf	Inf	-1
p	Inf	Inf	-1
q	Inf	Inf	-1
r	Inf	Inf	-1
s	0	4	-1
t	Inf	Inf	-1

			a	b	
			c	d	e
f	s	h	k	m	n
p	q	r	t	g	

A* search

A* Search: Example

	D	Key	Pred
a	4	8	c
b	Inf	Inf	-1
c	3	6	K
d	Inf	Inf	-1
e	Inf	Inf	-1
f	1	6	S
g	Inf	Inf	-1
h	1	4	S
k	2	4	h
m	Inf	Inf	-1
n	Inf	Inf	-1
p	Inf	Inf	-1
q	Inf	Inf	-1
r	Inf	Inf	-1
s	0	4	-1
t	Inf	Inf	-1

	D	Key	Pred
a	4	8	C
b	Inf	Inf	-1
c	3	6	K
d	Inf	Inf	-1
e	Inf	Inf	-1
f	1	6	S
g	Inf	Inf	-1
h	1	4	S
k	2	4	h
m	Inf	Inf	-1
n	Inf	Inf	-1
p	2	6	f
q	Inf	Inf	-1
r	Inf	Inf	-1
s	0	4	-1
t	Inf	Inf	-1

			a	b	
			c	d	e
f	s	h	k	m	n
p	q	r	t	g	

A* Search: Example

			a	b	
			c	d	e
f	s	h	k	m	n
p	q	r	t	g	

	D	Key	Pred
a	4	8	C
b	Inf	Inf	-1
c	3	6	K
d	Inf	Inf	-1
e	Inf	Inf	-1
f	1	6	S
g	Inf	Inf	-1
h	1	4	S
k	2	4	h
m	Inf	Inf	-1
n	Inf	Inf	-1
p	2	6	f
q	3	6	P
r	Inf	Inf	-1
s	0	4	-1
t	Inf	Inf	-1

	D	Key	Pred
a	4	8	C
b	Inf	Inf	-1
c	3	6	K
d	Inf	Inf	-1
e	Inf	Inf	-1
f	1	6	S
g	Inf	Inf	-1
h	1	4	S
k	2	4	h
m	Inf	Inf	-1
n	Inf	Inf	-1
p	2	6	f
q	3	6	P
r	4	6	Q
s	0	4	-1
t	Inf	Inf	-1

	D	Key	Pred
a	4	8	C
b	Inf	Inf	-1
c	3	6	K
d	Inf	Inf	-1
e	Inf	Inf	-1
f	1	6	S
g	Inf	Inf	-1
h	1	4	S
k	2	4	h
m	Inf	Inf	-1
n	Inf	Inf	-1
p	2	6	F
q	3	6	P
r	4	6	Q
s	0	4	-1
t	5	6	-1

	D	Key	Pred
a	4	8	C
b	Inf	Inf	-1
c	3	6	K
d	Inf	Inf	-1
e	Inf	Inf	-1
f	1	6	S
g	6	6	t
h	1	4	S
k	2	4	h
m	Inf	Inf	-1
n	Inf	Inf	-1
p	2	6	F
q	3	6	P
r	4	6	Q
s	0	4	-1
t	5	6	-1